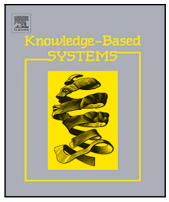
Knowledge-Based Systems 296 (2024) 111907

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys



A novel reinforcement learning based Heap-based optimizer

Xuesen Ma [a,b,*], Zhineng Zhong [a,c], Yangyu Li [c], Dacheng Li [c], Yan Qiao [a]

[a] School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, China
[b] Intelligent Manufacturing Institute of HFUT, Hefei, China
[c] Hefei Institutes of Physical Science, Chinese Academy of Sciences, China

ARTICLE INFO

Keywords:
Meta-heuristic algorithm
Optimization algorithm
Heap-based optimizer
Reinforcement learning

ABSTRACT

The Heap-Based Optimization (HBO), a novel meta-heuristic algorithm, constructs a hierarchical heap resembling a company structure to facilitate interaction among search agents. This enables the search agent to learn various levels of information from the population to obtain improved solutions, but it has drawbacks such as limited search capability and slow convergence speed when tackling complex optimization problems. To address these challenges, we propose an enhanced heap optimizer based on reinforcement learning(RLHBO), which demonstrates robust stability, rapid convergence, and high solution accuracy. Firstly, fixed operators are replaced with a reinforcement learning dynamic control search strategy in RLHBO to regulate the search range of agents. This prevents agents from being restricted in their search when encountering local or global optima, thus enhancing their search capability. Additionally, RLHBO introduces a dimensional self-learning strategy and a convergence strategy based on search direction similarity to address issues such as invalid and discrete solutions, accelerating convergence. Furthermore, an initialization strategy based on quasi-oppositional learning is employed to increase the diversity of the algorithm-generated population, surpassing the limitations of random initialization and further boosting convergence speed. Extensive testing on various complex functions from CEC2017 and CEC2022 demonstrates that RLHBO, as presented in this paper, outperforms HBO and numerous other advanced MAs in terms of search capability and convergence speed.

1. Introduction

In the last few decades, the continuous development of the economy and society has led to the emergence of increasingly complex optimization challenges across a variety of sectors, including engineering, healthcare, environmental science, and agriculture [1]. Scholars have traditionally employed deterministic methods to tackle these challenges [2], seeking optimal solutions within a defined search space by leveraging gradient information. However, as Abualigah et al. have pointed out, these methods often struggle with nonlinear, discontinuous, and multimodal optimization problems, frequently getting trapped in local optima [3,4]. The inherently nonlinear nature of many real-world optimization problems further limits the effectiveness of deterministic methods in a broad range of practical scenarios.

In response, researchers have begun to explore various meta-heuristic algorithms (MAs), which simulate phenomena and behaviors observed in nature and human society to iteratively find solutions without depending on gradient information [5]. These algorithms offer several advantages, including flexibility, simplicity, and the ability to find superior solutions more quickly than traditional methods. Meta-heuristic algorithms have demonstrated robustness and general applicability, effectively tackling nonlinear and discontinuous problems across diverse fields such as engineering, environmental science, and healthcare [6,7]. The remarkable performance of meta-heuristic algorithms, exemplified by Particle Swarm Optimization (PSO) [8], Genetic Algorithm (GA) [9], Differential Evolution (DE) [10], Grey Wolf Optimizer (GWO) [11], Biogeography-Based Optimization (BBO) [12], and others, has been significantly advanced. Nonetheless, the variety and complexity of real-world problems mean that no single algorithm can address all optimization challenges effectively, prompting the continuous proposal of new or improved MAs. The Heap Optimization Algorithm (HBO), developed by Askari et al. [13], represents a novel meta-heuristic algorithm inspired by the organizational structure of company personnel. It utilizes the heap data structure to simulate the hierarchy of a company's staff. This structure facilitates various levels of information interaction within populations, enabling HBO to exhibit superior performance in solving specific benchmark problems [14].

* Corresponding author at: School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, China.
E-mail addresses: mxs@hfut.edu.cn (X. Ma), 2022111115@mail.hfut.edu.cn (Z. Zhong), yyli@aiofm.ac.cn (Y. Li), dcli@aiofm.ac.cn (D. Li), qiaoyan@hfut.edu.cn (Y. Qiao).

https://doi.org/10.1016/j.knosys.2024.111907
Received 7 March 2024; Received in revised form 24 April 2024; Accepted 4 May 2024
Available online 8 May 2024

The application of HBO across several complex optimization challenges in different domains has been successful. For instance, in the field of engineering design, HBO has been applied to optimize parameters in photovoltaic systems, thereby significantly enhancing their energy conversion efficiency [15]. In electronic engineering, HBO has contributed to the optimization of filter design, leading to substantial improvements in performance [16]. Moreover, in the machine learning domain, HBO has been used to fine-tune parameters of Long Short-Term Memory (LSTM) models, improving the prediction accuracy of wind turbine power outputs [17]. These applications not only validate HBO's effectiveness in addressing complex issues but also showcase its broad utility and potential for future research and development. However, when addressing complex optimization problems, HBO still encounters the following issues that necessitate improvement: (i) **Insufficient search capabilities**. The search mechanism of HBO employs a fixed operator to adjust its search radius, which presents two major challenges. Firstly, when an agent is trapped in a local optimum, this fixed operator does not have the flexibility to expand the search radius according to the agent's current situation, impeding its ability to escape from the local optimum to explore more widely. Additionally, when an agent is near the global optimum, it fails to adequately reduce its search radius based on its position, leading to inadequate exploitation around the global optimum and significantly diminishing its search effectiveness. Furthermore, HBO's dependency on a random strategy for initializing the population leads to unstable quality in the initial population, negatively affecting the algorithm's search capabilities. (ii) **Slow convergence speed**. In the early stages of the search, the method used for updating an agent's position by self-interaction in each dimension frequently results in reproducing the same ineffective solution. Later in the search process, when the agent interacts with peers and direct leaders, it generates numerous discrete solutions that are significantly disparate from the overall population. These isolated solutions slow down the algorithm's convergence speed, presenting obstacles to achieving timely and effective optimization.

The exploration and exploitation challenges have long influenced the search capabilities of MAs. During exploration, the search agent traverses the global space to identify the region most likely to contain the global optimum solution. Conversely, during exploitation, the agent focuses its efforts on the area presumed to harbor the global optimum. Balancing these two phases is pivotal; a lack of balance may cause the agent to settle into a local optimum or lead to non-convergence [18].

With the advancements in Reinforcement Learning (RL) technologies, their application in MAs has seen increasing interest among researchers. The exploration and exploitation dilemma, a significant challenge impacting MAs' performance, underscores the key advantage of RL. Specifically, RL's strength lies in its ability to dynamically adjust strategies or parameters based on feedback from the current environment and tasks, thus facilitating an effective balance between exploration and exploitation [19,20]. This adaptability makes the algorithm more robust and capable of demonstrating enhanced performance on optimization problems across various fields. In contrast, methods relying on static strategies or parameters may encounter limitations due to their inability to adjust to environmental changes, leading to reduced performance or failure to fully exploit the algorithm's potential. Reinforcement learning's capacity for learning optimal strategies or parameters for different search environments has encouraged some scholars to apply RL in managing complex search strategies and parameter adjustments, thereby improving MAs' performance [21–23]. Results indicate that RL can reduce the uncertainty of manual control and increase the stability and efficiency of MAs.

Therefore, we propose a reinforcement learning-based heap optimization algorithm (RLHBO) to enhance the search capability and convergence performance of the HBO algorithm. To address HBO's limited search capabilities, we divide the agent into different search states and introduce a dynamic control strategy based on reinforcement learning. Specifically, we use the Q-learning algorithm to control state transitions. When the agent is in the expanded state, its search radius increases, enabling it to break out of local convergence zones and explore promising regions. Conversely, when in the reduced state, the search radius decreases, allowing for the exploitation of promising areas to locate optimal solutions. This approach ensures a balance between exploration and exploitation, thereby improving search capabilities.

Furthermore, to address the slow convergence issue of HBO in the early search stages, we introduce a dimensional self-learning strategy. This strategy allows the search agent to gather information about other dimensions of its solution vector, avoiding invalid solutions identical to the original solution. In later search stages, we implement a convergence strategy based on the similarity of search directions. Specifically, when the agent's search direction closely aligns with the population's optimal solution, indicating proximity to convergence, there is a higher chance of finding a global optimal solution between them. Thus, we enable the search agent to explore this area, facilitating convergence towards solving discrete solution problems and enhancing convergence speed.

Additionally, we employ the quasi-oppositional learning strategy to initialize the population. Specifically, the population is initialized by comparing quasi-oppositional learning with randomly generated candidate solutions, retaining the better-performing ones. This improves the quality of the initial population generated, thereby enhancing the algorithm's global optimization capability [24].

The main contributions of this paper are as follows:

- We propose a dynamic control search strategy based on reinforcement learning to dynamically adjust the search range of the current search agent through continuous learning and feedback, utilizing a set reward and punishment mechanism. This balances exploitation and exploration, enhancing the algorithm's ability to prevent falling into a local optimum.
- To expedite the algorithm's convergence, we address the issues of invalid and discrete solutions generated by HBO. Additionally, a dimensional self-learning strategy is introduced to tackle the problem of invalid solutions caused by HBO, and a convergence strategy based on search direction similarity is proposed to solve the discrete solution problem associated with HBO, thereby improving the algorithm's convergence speed and global optimization capabilities.
- The proposed RLHBO algorithm exhibits strong stability and fast convergence speed. Its performance on the standard test function sets CEC2017 and the latest CEC2022 surpasses that of the HBO algorithm and other advanced comparative MAs.

The rest of this paper is organized as follows: Section 2 introduces the original HBO, reinforcement learning, and related work. Section 3 discusses the search control strategy based on reinforcement learning, the accelerated convergence strategy, and the quasi-oppositional learning strategy for initializing the population proposed in this paper. Section 4 focuses on the benchmark test set, verifying and analyzing the performance of RLHBO. The final section, Section 5, summarizes the conclusions and outlines future steps.

2. Related work and preliminaries

2.1. Related work

2.1.1. HBO variants

Recent research on enhancing the HBO algorithm has been relatively limited, with the bulk of efforts directed towards its application in solving practical problems. These enhancement strategies predominantly aim to improve the solution's quality by refining the algorithm's location or initialization strategies. Basetti et al. proposed tackling HBO's local convergence issue by employing quasi-oppositional learning for generating random solutions, thus boosting the global search

capability of the algorithm [25]. Zhang et al. introduced a differential perturbation-based approach to strengthen the global search capacity of HBO by independently perturbing optimal, worst, and average individuals [26]. In an attempt to accelerate the convergence speed and enhance the capability to solve nonlinear planning problems, Rizk-Allah et al. utilized chaos theory to create initial solutions for the HBO algorithm [27]. Shaheen et al. focused on improving convergence speed and global search capabilities by allowing the agent to investigate the vicinity of the optimal individual within the population during the algorithm's later stages [28].

However, these strategies for improvement are static and lack adaptive adjustability. Moreover, they fail to tackle the critical challenge of balancing exploration and exploitation effectively, making them susceptible to local convergence and reducing global convergence speed.

2.1.2. Reinforcement learning for MAs

Reinforcement learning has recently demonstrated significant promise in addressing complex optimization challenges. To boost MAs' efficacy, numerous scholars have leveraged reinforcement learning for intricate search and parameter tuning strategies [29]. For instance, Luo et al. introduced a reinforcement learning-based method to adaptively refine the search agent throughout the iterative process, thereby augmenting the cuckoo search algorithm's efficiency in economic scheduling tasks [30]. In a similar vein, Li et al. employed reinforcement learning to modulate the neighborhood size of search particles, enhancing particle velocity updates and, consequently, the performance of the particle swarm optimization algorithm [1]. Additionally, Ghetas et al. applied reinforcement learning techniques to fine-tune the crawler search algorithm, specifically focusing on managing the balance between the exploitation and exploration phases upon encountering deteriorating search solutions [21]. Hu et al. utilized reinforcement learning to achieve an optimal balance between exploitation and exploration, optimizing the gray wolf optimization algorithm for feature selection challenges [31]. Meanwhile, Tian et al. implemented an operator selection approach within a differential evolutionary algorithm to address exploration and exploitation dilemmas in operator selection [32].

Contrasting with traditional search and parameter tuning approaches, reinforcement learning-based controls showcase remarkable adaptability. These strategies allow for dynamic adjustments of search parameters and strategies based on the agent's current status, effectively balancing exploration and exploitation, and thus elevating MAs' overall performance.

While existing MAs employ reinforcement learning to adjust specific search parameters or strategies to strike a balance between exploration and exploitation, these approaches are often designed with particular MAs in mind. Their parameter adjustment mechanisms tend to be complex or tailored to specific algorithms, limiting their general applicability. In contrast, our method innovatively leverages reinforcement learning to dynamically manage a common parameter among MAs: the search scope. This approach not only simplifies the control mechanism but also significantly enhances the algorithm's universality and adaptability. By focusing on the search scope as a universally applicable parameter, our method offers a direct yet effective way to balance exploration and exploitation, thereby optimizing the performance of various MAs. Moreover, by enabling search agents to adjust their search scope based on the current state, our strategy greatly improves search efficiency and adaptability. This makes the algorithms more capable of finding global optimal solutions in a wide range of search environments.

2.2. Preliminaries

2.2.1. HBO

The Heap-based Optimization Algorithm (HBO) is a Metaheuristic Algorithm (MA) inspired by human social structures, specifically utilizing a heap data structure to mirror the hierarchical organization of personnel within a company, as introduced by Askari et al. (2020) [13]. In this model, the heap's nodes are composed of key–value pairs, with the key representing the fitness of the search agent and the value indicating its index. This construction parallels the practice within companies where employees are positioned according to their capabilities, employing the fitness of the search agent as a basis for this arrangement. In the HBO algorithm, a fitness-based hierarchy is utilized to simulate the corporate personnel structure, where each search agent evaluates the quality of solutions based on the fitness function of the optimization problem. The higher the quality of the solution, the higher the level the individual occupies in the hierarchy. Individuals at higher levels are the direct leaders of those at the directly lower levels connected to them. Illustrated in Fig. 1, the root node symbolizes the population's optimal solution. The depicted heap structure, with a population size of 40, showcases a hierarchical distribution where one leader oversees three subordinates directly. The apex of this hierarchy is the lone top-performing individual, followed by a second level housing three individuals closely trailing the top performer, and so forth. The third level encompasses 9 individuals, while the fourth level includes 27 individuals. Within this context, a colleague denotes a node situated on the same hierarchical level as another individual, whereas a leader refers to a node's parent in the hierarchy. Every node, barring the root node, is part of a sibling group and has a parent node. Interactions within this structure, excluding the root node occupant who does not engage in location updates, are categorized into three types: self-interaction, interaction with a direct leader, and interaction with colleagues.

The HBO algorithm dictates the search agent's interaction strategy through a probabilistic approach, as delineated by Eqs. (1) and (2):

$$p_1 = 1 - t/T, \tag{1}$$

where p_1 signifies the likelihood of self-interaction, decreasing linearly as the algorithm progresses through its iterations t, with T being the total iteration count. The probability of engaging with a leader or colleagues is denoted by p_2, which is calculated as:

$$p_2 = p_1 + \left(1 - p_1\right)/2. \tag{2}$$

Here, p_2 exhibits a progressively increasing trend, indicating a shift in interaction preference as the algorithm iterates.

Self-interaction, depicted in Eq. (3), implies the search agent maintains its previous iteration's position information unaltered in the jth component:

$$X_i^j(t+1) = X_i^j(t). \tag{3}$$

Conversely, interaction with a leader, as defined in Eq. (4), updates the individual's location information to a position close to its direct leader, B^j, moderated by the parameters γ and λ:

$$X_i^j(t+1) = B^j + \gamma\lambda\left|B^j - X_i^j(t)\right|, \tag{4}$$

with γ oscillating between 0 and 2 in a triangular wave centered at 1, as specified in Eq. (5):

$$\gamma = |2 - 4mod(t, 25)/25|, \tag{5}$$

and λ being a scaling factor determined by a uniformly distributed random number r in [0, 1], as shown in Eq. (6):

$$\lambda = 2r - 1. \tag{6}$$

Interaction with colleagues, formulated in Eq. (7), allows for the update of the individual's location either towards a randomly selected colleague's position if said colleague's solution is superior or towards its current position otherwise:

$$X_i^j(t+1) = \begin{cases} S_r^j + \gamma\lambda|S_r^j - X_i^j(t)|, & \text{if } f(S_r) < f(X_i(t)); \\ X_i^j + \gamma\lambda|S_r^j - X_i^j(t)|, & \text{if } f(S_r) \geqslant f(X_i(t)); \end{cases} \tag{7}$$

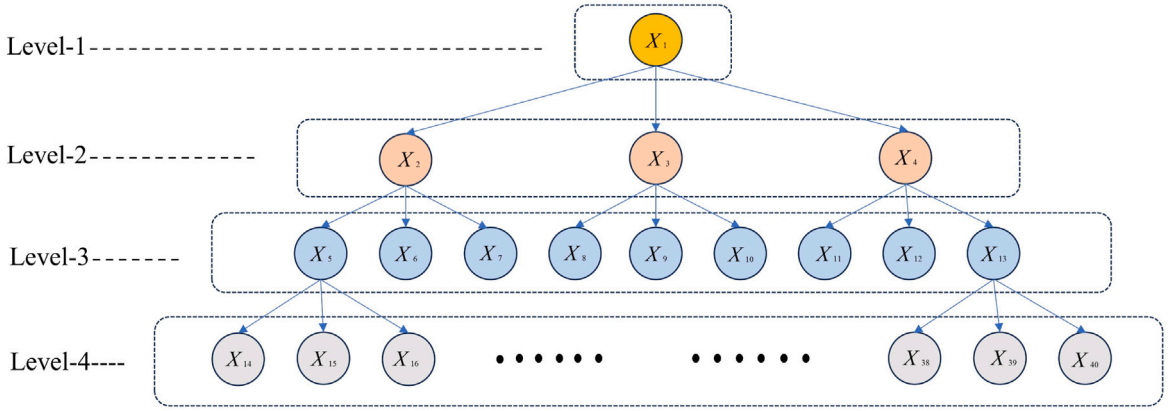


Fig. 1. Partial examples of ternary heap. (Nodes at the same level are colleagues, and nodes directly connected to the upper level are direct leaders.)

The algorithm leverages the generated random probability p to decide on the interaction mode:engaging with direct leaders if $p > p_1$, and with colleagues if $p > p_2$. The procedural logic of these interactions is encapsulated in the HBO algorithm's pseudo-code, as referenced in Algo. 1. The HBO algorithm's interaction strategy differentiates it from other MAs. However, since it is a recent proposal and has not yet received widespread attention from scholars, there is significant scope for improving its performance.

Algorithm 1 HBO algorithm

Require: N: population size,
D: problem dimension,
T: maximum number of iterations
Initial the population randomly
Build a heap
for $t = 1 : T$ **do**
  Set $p = rand$ p_1 and p_2 by Eq. (1) and Eq. (2)
  if $p < p_1$ **then**
    Compute agent's position by Eq. (3)
  else if $p < p2$ **then**
    Compute agent's position by Eq. (4)
  else
    Compute agent's position by Eq. (7)
  end if
  Update population by compare f_{new} with f_{old}
  Update the heap [13]
end for
Ensure: Optimal solution

2.2.2. Reinforcement learning

Reinforcement learning is a crucial branch of machine learning. It can maximize long-term cumulative benefits through continuous trial and error and interaction between the agent and the complex environment, enabling the agent to make optimal decisions based on the current state. The powerful decision-making ability of reinforcement learning has made this technology widely used in important fields such as intelligent control and robotics [33,34].

Reinforcement learning is an essential branch of machine learning that aims to maximize long-term cumulative rewards through ongoing trial and error and interaction between the agent and a complex environment. This process allows the agent to make optimal decisions based on the current state. The significant decision-making capability of reinforcement learning has led to its wide application in key fields such as intelligent control and robotics [33,34].

In reinforcement learning, Q-values are crucial, representing key–value pairs of different states and actions. The framework consists of five components: the agent, the environment, the state space, the action space, and the reward. The state space includes all possible states within the environment, while the action space consists of all available actions for the intelligent agent.

Q-learning, a model-free method, is noted for its application versatility. This paper utilizes Q-learning to improve the search strategy of the HBO algorithm, thereby enhancing its performance. In Q-learning, the agent determines the Q-value based on the environment's current state using a Q-table, to take the most suitable action for the current state and maximize cumulative gain. The Q-table is updated continuously as the state of the environment changes, according to the following equation:

$$Q_{(t+1)}\left(s_t, a_t\right) \leftarrow Q_t\left(s_t, a_t\right) + \alpha\left[r_{t+1} + \gamma \operatorname{Max} Q_{t(s_{t+1}, a)} - Q_t\left(s_t, a_t\right)\right]. \tag{8}$$

where s_t and s_{t+1} are the current and next agent states, r is the real-time reward received by performing action a in state s_t, α is the learning rate, γ is the discount factor, and the Q-value is the cumulative reward achieved by the agent performing action a in state s_t.

3. Proposed algorithm

To enhance the search capability and convergence speed of the HBO, this paper introduces a Reinforcement Learning-based Heap Optimization algorithm (RLHBO) aimed at addressing the following challenges:

(1) The search range of the search agent significantly influences the performance of MAs. While HBO employs the fixed operators defined in Eqs. (5) and (6) for computations, these operators tend to confine the search within local optima. Consequently, the search agent struggles to adaptively expand its search radius, often resulting in premature convergence to local optima without achieving the global optimum. Furthermore, when nearing the global optimum, the agent fails to dynamically reduce its search radius, adversely affecting the convergence speed.

(2) In HBO, the mechanisms for exploration and exploitation rely on the probability parameters specified in Eq. (1). Initially, there is a pronounced tendency to select self-interaction updates to promote exploration. However, this approach leads to generating positions identical to those in the previous iteration, rendering them ineffective solutions. In the search's later stages, the likelihood increases for interactions with peers or direct interactions with the leader. Unfortunately, the solutions derived from these interactions hinder population convergence and result in dispersed solutions that significantly slow down the HBO algorithm's convergence rate.

(3) The HBO algorithm's approach to population initialization involves randomization, which can result in uneven population distribution and reduced diversity within the population, subsequently impacting the algorithm's convergence speed.

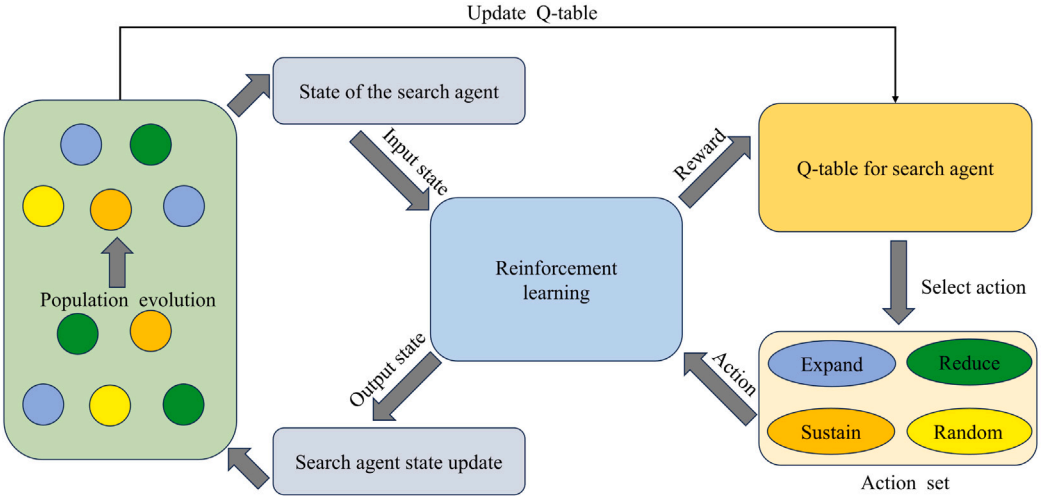


Fig. 2. Reinforcement learning strategy for search control.

To address the identified issues with the HBO, this paper proposes three strategies aimed at enhancing the algorithm:

(1) Reinforcement learning strategy for search control: We utilize a reinforcement learning strategy to regulate the search range of the search agent. This innovative approach allows the algorithm to dynamically adjust the search radius based on the search agent's current state, facilitating the avoidance of local optima and the achievement of the global optimum. This approach ensures a balance between exploration and exploitation, thereby improving search capabilities.

(2) Dimensional self-learning strategy and convergence strategy based on search direction similarity: We introduce a dimensional self-learning strategy to tackle the problem of generating invalid solutions in the algorithm's early stages. Furthermore, we propose a convergence strategy based on search direction similarity to improve the algorithm's convergence speed and global optimization capabilities in the later stages. These strategies collectively contribute to accelerating the convergence process and convergence strategy based on search direction similarity enhancing the algorithm's ability to find the global optimum.

(3) Quasi-oppositional learning for initial population generation: To improve the quality of initial population generation, we initiate the HBO algorithm's population using a quasi-oppositional learning strategy. This method involves generating new candidate solutions, thereby enhancing the convergence speed of the HBO algorithm and its capability to reach the global optimal solution more effectively.

The reinforcement learning strategy for search control is elaborated in Section 3.1. The strategies for speeding up convergence through dimensional self-learning and convergence strategy based on search direction similarity are discussed in Section 3.2. Finally, the application of quasi-oppositional learning for population initialization is detailed in Section 3.3.

3.1. Reinforcement learning strategy for search control

In the field of MAs, especially when addressing complex optimization challenges, achieving a balance between exploitation and exploration is crucial. The HBO algorithm uses a fixed operator, as shown in Eqs. (5) and (6), to adjust the search range of the agent. However, this method often leads to the issue of agents getting trapped in local optima, which in turn slows down the convergence process. To overcome this limitation, our paper introduces a search control strategy that employs reinforcement learning to dynamically adjust the search range, as illustrated in Eqs. (9) and (10). This strategy enhances the algorithm's capability to maintain an effective balance between exploitation and exploration, thereby improving its overall search performance.

$$X_i^j(t+1) = B^j + r_i(t+1)\left|B^j - X_i^j(t)\right|. \tag{9}$$

$$X_i^j(t+1) = \begin{cases} S_r^j + r_i(t+1)|S_r^j - X_i^j(t)|, \\ (if\ f(S_r) < f(X_i(t))); \\ X_i^j + r_i(t+1)|S_r^j - X_i^j(t)|, \\ (if\ f(S_r) \geqslant f(X_i(t))); \end{cases} \tag{10}$$

The proposed reinforcement learning-based search control strategy redefines and meticulously designs the foundational components of reinforcement learning–agents, environments, states, actions, and rewards–to better suit the optimization context:

- Environment: the search space of the optimization problem.
- Agent: each individual in the HBO population acts as an agent.
- State: the search agents are classified into four states based on the search range: increasing search radius state, decreasing search radius state, search radius unchanged state, and search radius reset randomly state.
- Action: alters the current search state of the search agent.
- Reward: set rewards based on the quality of new solutions obtained by the search agent.

The RLHBO algorithm utilizes a Q-learning-based method to dynamically control the search radius of the search agent. The learned Q-table is utilized to enable the agent to identify the optimal search strategy based on its current state. In this framework, each search agent in the HBO algorithm functions as an independent agent with its own Q-table. The population size is maintained as in the original HBO algorithm, consisting of 40 distinct Q-tables. The calculation of the q-value follows the update rule presented in Eq. (8). The learning rate, α, is a critical parameter in the Q-learning algorithm that balances the incorporation of new information with the existing knowledge during the Q-value update. When α is close to 1, the update is heavily influenced by newly acquired information, whereas an α closer to 0 means the update relies more on previously learned information. Given the initial scarcity of information, the learning rate α should decrease progressively as more information is accumulated. Consequently, this paper implements an adaptive learning rate α, as specified in Eq. (11).

$$\alpha(t) = 1 - \left(0.9 \times \frac{t}{T}\right). \tag{11}$$

This adaptive approach to the learning rate ensures a balanced update of the Q-value, reflecting both the depth of the agent's accumulated knowledge and the integration of new insights, thereby

enhancing the strategic adaptation of the search process within the RLHBO framework (see Fig. 2).

3.1.1. State and action

In the RLHBO framework, individuals in the population are classified into four distinct states based on adjustments to their search radius: *Expand*, *Reduce*, *Sustain*, and *Random*. The specific parameter settings of the individuals in the population are shown in Eq. (12), where r_i represents the search radius of the search agent and r_{scale} represents the search radius scaling factor. After experiments, setting r_{scale} to 1.05 has the best effect.

$$r_i(t+1) = \begin{cases} r_i(t) \times r_{scale}, & if\, state = Expand; \\ r_i(t)/r_{scale}, & if\, state = Reduce; \\ r_i(t), & if\, state = Sustain; \\ rand(), & if\, state = Random; \end{cases} \tag{12}$$

- In the *Expand* state, the search agent will increase the search radius to avoid falling into a local optimum, thus enhancing the exploration capability of the search agent.
- In the *Reduce* state, the search agent will reduce the search radius so that it searches in the vicinity of the learned individual, thus enhancing the exploitation of the individual as well as the speed of convergence.
- In the *Sustain* state, the current search agent will keep the previous state unchanged, thus ensuring the stability of the searching individual.
- In the *Random* state, the search agent sets its search radius to a random value between [0, 1], thus allowing the individual to jump out of the local optimum point and thus find a better solution.

The action changes the search state of the search agent. The state is selected using the ϵ-greedy strategy. There is an ϵ probability of selecting the action with the highest Q value in the current state, and a probability of $1-\epsilon$ to perform random actions, as shown in Eq. (13).

$$\text{action}_{\text{next}} = \begin{cases} \text{Argmax}\left[Q(l_C, \text{action}_C)\right], \\ (if\ \text{rand}\ () < \epsilon); \\ \text{action}_{\text{random}}, \text{otherwise}; \end{cases} \tag{13}$$

In the early stages of the search process, the information in the Q table may not be sufficient to guide the search agent to make the best decision. Therefore, the value of ϵ should be set higher in the early stages and gradually decreased as the search progresses. It is set by Eq. (14). Where $c \in [0, 1]$, t is the current number of iterations, and T is the total number of iterations.

$$\epsilon = c \times \frac{t}{T}. \tag{14}$$

3.1.2. Reward

In the RLHBO, the search agent adjusts the search radius based on the magnitude of the Q-values of the three interaction strategies in the current state. After adjusting the search radius of an individual using Eq. (13) (i.e., the ϵ-greedy strategy), the search agent receives a corresponding reward. The reward is given to the search agent using a fitness function and an evolutionary factor. The evolution factor is a metric used to assess population diversity in MAs.

$$R = \begin{cases} 2, if & f(x_{new}) < f(x_{old}) \quad and \\ & E_f(x_{new}) > E_f(x_{old}) \\ 1, if & f(x_{new}) < f(x_{old}) \quad and \\ & E_f(x_{new}) \le E_f(x_{old}) \\ 0, if & f(x_{new}) \ge f(x_{old}) \quad and \\ & E_f(x_{new}) > E_f(x_{old}) \\ -2, if & f(x_{new}) \ge f(x_{old}) \quad and \\ & E_f(x_{new}) \le E_f(x_{old}) \end{cases} \tag{15}$$

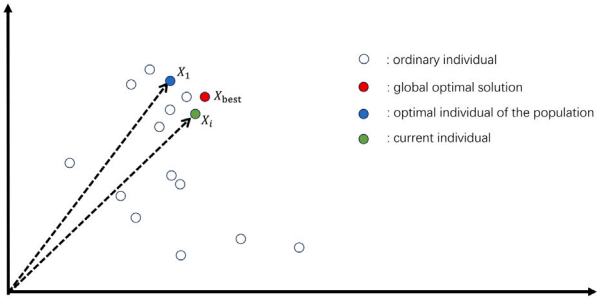


Fig. 3. Cosine similarity to judge similarity.

The reward is calculated according to Eq. (15) for minimization problems. A positive reward is given when a search agent acquires a new solution through interaction that is superior to the original, whereas a negative penalty is applied in the opposite scenario.

Where E_f represents the evolution factor, used to assess the population diversity of the meta-heuristic algorithm. A higher evolution factor indicates greater population diversity, which aids the meta-heuristic algorithm in avoiding local optima and improves its global search capability. Thus, if the new solution discovered by the current search agent exceeds the evolutionary factor of the original solution, an additional reward is warranted. Conversely, a penalty is appropriate if it is lesser. The metric of E_f is also employed in literature such as [1,35] to evaluate population diversity. E_f is determined by the following equations: Eqs. (16) and (17).

$$d_i = \frac{1}{N-1} \sum_{j=1, j\neq i}^{N} \sqrt{\sum_{k=1}^{D} \left(x_i^k - x_j^k\right)^2}. \tag{16}$$

$$E_f(i) = \frac{d_i - d_{\min}}{d_{max} - d_{min}}. \tag{17}$$

where N represents the population size, and D represents the dimension of the search space of the problem to be optimized. d_i represents the average distance of the individual from the rest of the population, d_{max} represents the maximum distance between the first individual and the rest of the population, and d_{min} represents the shortest distance in the population.

3.2. Speed up convergence

The presence of invalid solutions and discrete solution issues in the HBO significantly impacts the convergence speed of the algorithm. In Eqs. (1) and (2), it is shown that during the early search stages, the HBO algorithm has a higher probability of choosing self-interaction, while in the late search phase, the HBO has a higher probability of interacting with its direct leader as well as with its colleagues. During self-interaction, if the location update strategy of each dimension of the search agent involves self-interaction, the search agent will simply retain the solution generated in the last iteration. This can result in the original and new solutions being the same, which can lead to the production of invalid solutions. This affects the algorithm's ability to search globally and reduces the speed of convergence. In the later stages of the search, the HBO algorithm's search agent interacts with colleagues and superiors, increasing the population's diversity and enhancing its ability to escape local optima. However, this also results in a large number of discrete solutions scattered throughout the search space, which reduces the algorithm's convergence speed.

Therefore, to enhance the convergence speed of the algorithm, this paper proposes a dimensional self-learning strategy and a convergence strategy based on search direction similarity.

In the early stages of HBO, we propose a dimensional self-learning strategy to enhance HBO's self-interaction method to solve the problem

of invalid solutions. The specific principle involves randomly selecting a dimension value from the current individual and using it as the value for another dimension of the same individual. This allows individuals to access information from other dimensions, reducing the ineffective solutions produced by the algorithm. The mathematical model is represented by the following Eq. (18):

$$X_i^j(t+1) = X_i^k(t) \ (\ t < 0.05T). \tag{18}$$

where $k \in [1, D]$ is the randomly selected dimension. To solve the discrete solution problem that arises in the later stages of the search process, we propose a convergence strategy based on the similarity of search directions. Cosine similarity and Euclidean distance are commonly used to evaluate the similarity between two vectors in high-dimensional space. Research in [36] shows that in complex high-dimensional spaces, it is best to evaluate the difference between two vectors in direction. As shown in Fig. 3, this paper utilizes cosine similarity $\cos(\theta)$ to assess the similarity between the current individual X_i and the optimal individual X_1 of the population; if $\cos(\theta)$ is larger, the difference between the two position vectors is smaller. In the later stage of the search, when the cosine value is close to 1 between X_i and X_1, it indicates that the population is nearing the convergence state. There is a high likelihood that the global optimal solution X_{best} exists between X_i and X_1. The mathematical model is as follows: Eq. (20).

$$\cos(\theta) = \frac{\sum_{i=1}^{n}\left(X_i * X_1\right)}{\sqrt{\sum_{i=1}^{n}\left(X_i\right)^2} * \sqrt{\sum_{i=1}^{n}\left(X_1\right)^2}}. \tag{19}$$

$$X_i^j(t+1) = X_{best}^j(t) + \left|X_{best}^j(t) - X_i^j(t)\right| \times r_i(t+1), \quad (t > 0.75T \&\& \cos(\theta) > 0.85). \tag{20}$$

where $r_i(t+1)$ represents the current search radius of the search agent, which is adaptively adjusted by the reinforcement learning dynamically controlled search strategy.

3.3. Quasi-oppositional learning

In scenarios lacking prior knowledge about the search space, many Metaheuristics (MAs) default to a purely random strategy for populating their initial population. This approach, while straightforward, often culminates in a disproportionate distribution of solutions and diminished population diversity, adversely affecting the algorithm's efficiency.

Oppositional learning emerges as an effective optimization technique within MAs, providing a robust strategy for population initialization. Its fundamental concept revolves around creating solutions that are diametrically opposite to the initial ones and then evaluating their efficacy. By selecting the more favorable solutions for the MA's initialization phase, this method substantially boosts the diversity of the initial population. Such enhancement is crucial for avoiding premature convergence on local optima and accelerating the algorithm's progression towards identifying the global optimum [37].

Oppositional learning methods have been acknowledged for their versatility and effectiveness in optimization processes. Research has shown that solutions generated through backward learning tend to have a higher probability of reaching the global optimum than those produced entirely at random [38]. Let $X_i = [X_1, X_2, \ldots, X_D]$ be a solution where $X_i \in [L_D, U_D]$ in the D-dimensional search space, and the opposite solution of X_i is $X_i = L_i + U_i - X_i$, where $L_i = [L_1, L_2, \ldots, L_D]$ and $U_i = [U_1, U_2, \ldots, U_D]$ are the maximum and minimum values of the boundary of the search space.

To enhance the efficacy of oppositional learning methods, significant research has been conducted. Among the noteworthy advancements is Quasi-oppositional learning, introduced by Rahnamayan et al. [24]. This technique aims to improve the population quality beyond what traditional oppositional learning achieves. Due to the significant positional shifts in solutions generated by oppositional learning, there is a tendency for these solutions to have lower precision, often resulting in stagnation at local optima [39]. Conversely, solutions produced through quasi-oppositional learning undergo smaller positional changes, which can enhance the precision of the generated solutions. Quasi-oppositional learning has been shown to increase the accuracy of generated solutions significantly. Numerous scholars have adopted quasi-oppositional learning to bolster the performance of their algorithms [25,39]. The solution x_i^{qo} generated by quasi-oppositional learning is shown in Eq. (21).

$$x_i^{qo} = \text{rand}\left(\frac{lb_i + ub_i}{2}, lb_i + ub_i - x_i\right). \tag{21}$$

x_i is a solution where $x_i \in [lb_i, ub_i]$ of the d-dimensional search space. lb_i and ub_i represent the lower and upper boundaries of the search space, respectively.

Since quasi-oppositional learning has a greater probability of obtaining the global optimal solution, this paper uses it to initialize the population. Specifically, the candidate solutions generated by quasi-oppositional learning are compared with the original randomly generated solutions. The best-performing solution among these options is selected as the initialization solution for the population to enhance diversity and improve the ability to obtain the global optimal solution.

To further explain in detail the principle of the RLHBO algorithm proposed in this paper, its pseudo-code is as Algo. 2.

Algorithm 2 RLHBO algorithm

Require: N: population size,
D: problem dimension,
T: maximum number of iterations
Generate initial QQL population X
Initial the states, actions,and other parameters in RL
Build a heap
for $t = 1 : T$ **do**
  Set the state of the agent via Eq. (13)
  Change $r_i(t+1)$ by Eq. (12)
  Set $p = rand$ and p_1 and p_2 by Eq. (1) and Eq. (2)
  if $p < p_1$ **then**
    if $t < 0.05T$ **then**
      Compute agent's position by Eq. (18)
    else
      Compute agent's position by Eq. (3)
    end if
  else if $p < p2$ **then**
    if $t > 0.75T$ & $cos(\theta) > 0.85$ **then**
      Compute agent's position by Eq. (20)
    else
      Compute agent's position by Eq. (9)
    end if
  else
    if $t > 0.75T$ & $cos(\theta) > 0.85$ **then**
      Compute agent's position by Eq. (20)
    else
      Compute agent's position by Eq. (10)
    end if
  end if
  Update population by compare f_{new} with f_{old}
  Compute reward by Eq. (15)
  Update Q-Table by Eq. (8)
  Update the heap
end for
Ensure: Optimal solution

Table 1
Detail of the state-of-the-art comparison algorithms.

Algorithm	Years	Parameter information
MPA [40]	2020	$N = 25, FADs = 0.2, P = 0.5$
CGO [41]	2021	$N = 50, R = [0, 1], \delta = [0, 1], \epsilon = [0, 1]$
BWO [42]	2022	$N = 100, W_f = [0.05, 0.1]$
CEOA [43]	2023	–
COA [44]	2023	–
MPSO [45]	2020	$N = 50, w = 1, \varphi_1 = \varphi_2 = 2.5$
IGWO [46]	2021	$N = 30, a = [0, 2]$
HBO [13]	2020	$N = 40, degree = 3$
HODEI [28]	2022	$N = 40, degree = 3$
RLHBO	Presented	$N = 40, degree = 3$

3.4. Computational complexity analysis

The space complexity of HBO is $O(N_a D)$, and in RLHBO, the additional space complexity primarily stems from the reinforcement learning control strategy, specifically the space complexity for storing the Q-tables for each search agent: $O(16N_a)$. Thus, the space complexity of RLHBO is $O((16 + D)N_a)$, where D represents the dimensionality of the search space, and N_a denotes the total number of search agents in the population.

The time complexity of HBO is shown in Eq. (22) [13]:

$$T(N) = \begin{cases} O(I N_a D), & D > T_f; \\ O\left(I N_a T_f\right), & \text{Otherwise}; \end{cases} \tag{22}$$

where I represents the maximum number of iterations, and N_a denotes the population size. Compared to the HBO algorithm, the RLHBO algorithm introduces additional time costs associated with three strategies. The initialization strategy is executed only once at the beginning of the algorithm, and its time complexity is $O(N_a D)$, which can be negligible compared to the overall time consumption of the algorithm. Moreover, the step of calculating cosine similarity in the acceleration convergence strategy has a time complexity of $O(I N_a T_{cos})$, where T_{cos} is the time required to calculate cosine similarity. The main time cost of the reinforcement learning control strategy is $O(I N_a T_d)$, where T_d is the time required to calculate population diversity. T_{cos} involves calculating the cosine similarity, which requires computation across each dimension for the dot product and norms. The complexity of this calculation is $O(3D)$, inherently greater than D. T_d involves calculating the average distance between each individual and all others in the population. The time complexity for calculating the distance between one individual and the others is $O(D)$. Given that this calculation is performed $N_a - 1$ times, the overall time complexity for T_d is $O(N_a D)$, which is also much greater than D. The time complexity of RLHBO is: $O(I N_a(T_{cos} + T_d + T_f))$.

4. Experimental results and discussions

To evaluate the effectiveness of RLHBO as proposed in this article for solving complex problems, we conducted numerous experiments using the mainstream CEC2017 and the latest CEC2022 benchmark test sets to compare its performance with other advanced MAs. To verify the effectiveness of the improvement strategy proposed in this article, a comparison was made at CEC2017 with algorithms using the three improvement strategies proposed in this paper;

The results show that this article fully verifies the effectiveness of the proposed improvement strategy. The solution accuracy, stability, and convergence speed of RLHBO are better than those of HBO and other advanced comparative MAs.

4.1. Experimental setup

To assess the performance of the RLHBO algorithm, we conducted several experiments on functions of varying dimensions in the CEC2017 and CEC2022 benchmark test sets. The CEC2017 includes four types of functions: F1–F3 for unimodal functions, F4–F11 for multimodal functions, F12–F20 for hybrid functions, and F21–F30 for complex functions. A detailed description of the benchmark functions can be found in Ref. [47]. The CEC2022 comprises 12 functions, with F1 is an unimodal function, F2–F5 are multimodal functions, F6–F8 are hybrid functions, and F9–F12 as are complex function. The details of these functions can be found in Ref. [48]. The unimodal function is generally used to evaluate the algorithm's exploitation ability, the multimodal function is used to test the algorithm's exploration ability, and the complex function is used to test the algorithm's ability to solve complex problems. To more effectively evaluate the performance of the proposed RLHBO algorithm, this paper selects several representative state-of-the-art algorithms for comparison. The selected comparison algorithms are mainly classified into three categories. The details of these algorithms are shown in Table 1.

- Recently proposed novel meta-heuristic algorithm: MPA (Marine Predators Algorithm) [40], CGO (Chaos Game Optimisation) [41], BWO (Beluga whale optimisation) [42], CEOA (officer election optimisation algorithm) [43], COA (Coati Optimisation Algorithm) [44];
- Recently proposed algorithm based on an improvement of the classical algorithm: MPSO (Motion-encoded particle swarm optimisation) [45], IGWO (Improved Grey Wolf Optimizer) [46];
- Related algorithms for HBO: HBO [13], HODEI (Heap-based Optimizer with Deeper Exploitative) [28];

To further validate the RLHBO algorithm's efficacy, we conducted comparative tests with other leading algorithms on the CEC-2017 benchmark set across dimensions of 10, 30, and 50. For these tests, the maximum number of objective function evaluations was set to 50,000 for D = 10, 300,000 for D = 30, and 500,000 for D = 50. Data for the CEOA algorithm was sourced from [43], while the COA algorithm's data was obtained from [44]. Furthermore, to further assess the stability of RLHBO, we also conducted evaluations using the 10-dimensional and 20-dimensional benchmarks from the latest CEC2022 benchmark suite. We set the maximum number of function evaluations to 50,000 for 10-dimensional functions and 100,000 for 20-dimensional functions. Consistently across all benchmarks, every algorithm was independently run 31 times on each benchmark. The outcomes of these tests are documented in Table 2, Table 3, Table 4, and Table 5, detailing the minimum function values achieved by each algorithm, along with their standard deviations and rankings for a comprehensive comparison.

4.2. Performance comparison on CEC2017 test set

RLHBO demonstrates superior performance over HBO across all 29 10-dimensional functions tested in CEC2017. Moreover, it demonstrates a strong competitive advantage over other state-of-the-art algorithms, securing the top rank overall with an average ranking of 1.62. RLHBO outperforms in 20 functions, while the second-ranked MPA excels in only 4 functions. In CEC2017, RLHBO achieved an average ranking of 1.69 in 30-dimensional functions and 1.86 in 50-dimensional functions, securing the top position overall. These results underscore RLHBO's exceptional performance across 10, 30, and 50-dimensional functions of CEC2017, surpassing both HBO and other comparative algorithms. This affirms its potent capability to solve complex optimization problems.

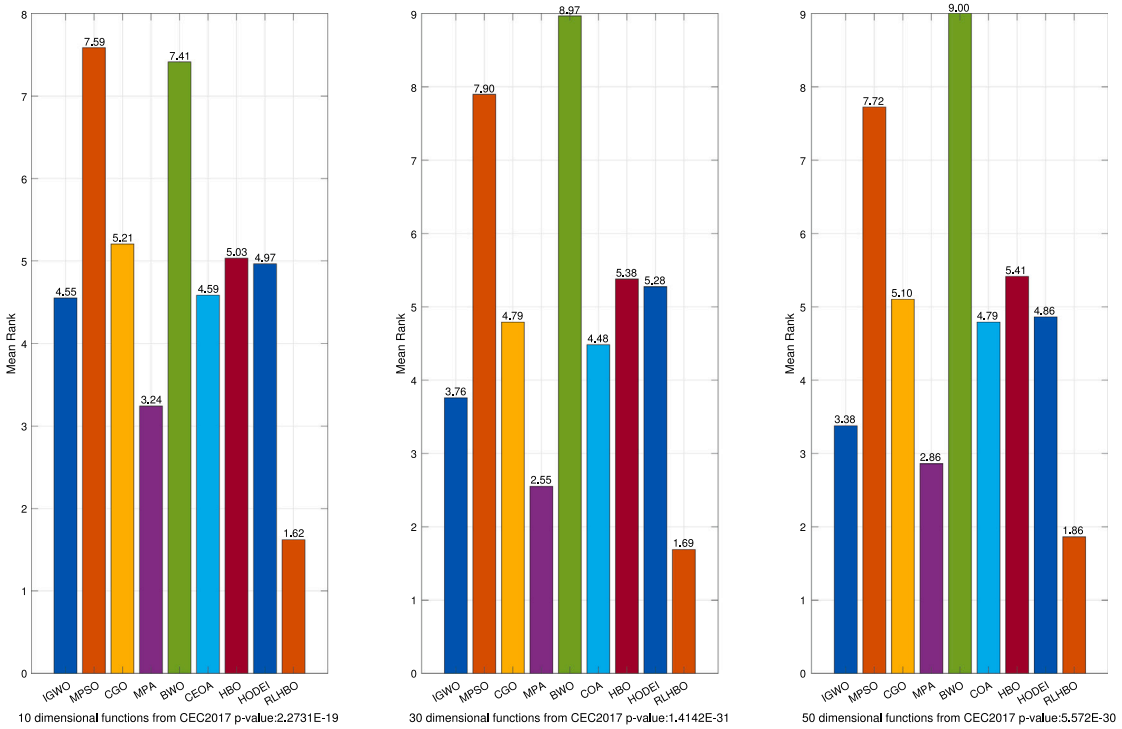


Fig. 4. Result of Friedman Mean Rank Test for functions from CEC2017.

4.3. Performance comparison on CEC2022 test set

To further validate the effectiveness of the RLHBO algorithm in solving complex problems, we compare it with HBO, MPA, and IGWO, which are ranked second only to RLHBO in terms of performance in CEC2017. In this study, we evaluate the algorithms using the latest CEC2022 test functions.

From Table 5, it is evident that RLHBO performs optimally in both the 10-dimensional and 20-dimensional functions of CEC2022, achieving first average rankings of 1.73 and 1.50, respectively. Additionally, RLHBO outperforms 7 out of 10 functions in the 10-dimensional function and 6 out of the 20-dimensional functions, representing a significant improvement over HBO and other comparative algorithms. These results reaffirm the outstanding ability of RLHBO to solve complex optimization problems, as stated in the paper.

4.4. Statistical analysis

To evaluate the distinct performance advantage of the RLHBO algorithm over others, this section employs both Friedman's test and the Wilcoxon signed-rank test for a comprehensive analysis. We derived our performance comparison from the CEC2017 benchmarks across dimensions of 10, 30, and 50, with the outcomes presented in Fig. 4. The results of the Wilcoxon signed-rank tests, focusing on the complex functions within CEC2017, are detailed in Table 6, Table 7, and Table 8. The Friedman test, a non-parametric statistical approach, is applied to determine if there are significant differences in performance among several algorithms [49]. A p-value below the conventional significance threshold of 0.05 indicates a substantial performance disparity. The findings from the Friedman test, illustrating how RLHBO compares to other algorithms, are depicted in Fig. 4.

In the detailed analysis of the CEC2017 10-dimensional function, the p-value stands at an extremely low 2.2731E−19, far beneath the standard threshold of 0.05, confirming a significant disparity. This result highlights the RLHBO algorithm's distinct advantage over its competitors. Specifically, RLHBO leads with the smallest average ranking of 1.62, closely followed by MPA at 3.24, IGWO at 4.55, and others, illustrating RLHBO's superiority in this dimension. Notably, it significantly surpasses HBO, which has an average ranking of 5.03, along with other algorithms.

Moving to the 30-dimensional function within CEC2017, the p-value is astonishingly low at 1.4142E−31, underscoring a significant difference in algorithm performance. In this scenario, RLHBO maintains its leading position with an average ranking of 1.69, outperforming other algorithms such as MPA, IGWO, and notably HBO, which secures a ranking of 5.38. This demonstrates RLHBO's consistent performance advantage across varying dimensions.

For the 50-dimensional function, the findings are equally compelling with a p-value of 5.572E−30, indicating significant performance differentiation. RLHBO again leads with the smallest average ranking of 1.86, followed by MPA and IGWO among others. This solidifies RLHBO's status as the top-performing algorithm across all tested dimensions of the CEC2017, clearly outpacing HBO and the rest of the compared algorithms.

These analyses conclusively affirm that RLHBO not only significantly differs from but also consistently outperforms the other algorithms across the 10-dimensional, 30-dimensional, and 50-dimensional functions of the CEC2017 benchmarks.

To further substantiate the performance distinctions between RLHBO and other benchmarked algorithms, we employed the Wilcoxon signed-rank test for a detailed comparative analysis. The Wilcoxon signed-rank test, a non-parametric statistical method cited from Derrac et al. (2011), prioritizes the performance metrics of algorithms to ascertain the significance of their differences [49]. This approach generates rankings based on how each algorithm performs relative to the others, offering a clear view of their comparative strengths and weaknesses.

The outcomes of the Wilcoxon rank test, as applied to the 10, 30, and 50-dimensional functions of CEC2017, are systematically tabulated. Within this framework, for RLHBO, R^+ denotes the aggregate rankings of the test functions where RLHBO outperforms its counterparts. Conversely, R^- signifies the cumulative rankings of instances where RLHBO is outperformed by the comparison algorithms. Situations where RLHBO and another algorithm achieve identical performance results in the averaging of their rankings, thus reflecting in

Table 2
Optimization results and comparison for CEC-2017 test functions (the dimension $m = 10$).

Function	Metric	IGWO	MPSO	CGO	MPA	BWO	CEOA	HBO	HODEI	RLHBO
F1	mean	1.3326e+04	1.1351e+09	1.0000e+02	1.0394e+02	4.3215e+07	1.6762e+02	1.0408e+03	4.2046e+02	**1.0000e+02**
	std	5.0609e+03	5.6753e+08	1.3631e−07	6.8756e+00	2.0149e+07	1.7872e+02	1.1878e+03	3.0820e+02	1.6534e−14
	Rank	7	9	2	3	8	4	6	5	**1**
F3	mean	3.0000e+02	3.0000e+02	**3.0000e+02**	3.0000e+02	7.3451e+02	3.0000e+02	3.0000e+02	3.0002e+02	3.0000e+02
	std	5.9319e−04	1.4410e−11	3.2819e−14	3.1246e−06	4.3891e+02	3.0000e−02	1.7455e−03	4.7690e−01	7.3440e−13
	Rank	7	3	**1**	4	9	5	6	8	2
F4	mean	4.0220e+02	4.5011e+02	4.1391e+02	4.0175e+02	4.1115e+02	4.0240e+02	4.0484e+02	4.0385e+02	**4.0000e+02**
	std	1.5613e+00	4.7468e+01	2.7810e+01	4.6467e−01	1.2797e+00	1.1564e+00	3.8721e−01	1.2232e+00	5.1950e−06
	Rank	3	9	8	2	7	4	6	5	**1**
F5	mean	**5.0521e+02**	5.2137e+02	5.0572e+02	5.1069e+02	5.2203e+02	5.0719e+02	5.0929e+02	5.0956e+02	5.0650e+02
	std	2.4560e+00	7.6948e+00	2.2062e+00	1.4925e+00	3.6794e+00	3.3000e+00	3.9544e+00	4.3853e+00	2.1320e+00
	Rank	**1**	8	2	7	9	4	5	6	3
F6	mean	6.0000e+02	6.0074e+02	6.0019e+02	6.0000e+02	6.0502e+02	**6.0000e+02**	6.0000e+02	**6.0000e+02**	**6.0000e+02**
	std	8.3967e−04	5.8663e−01	2.2095e−01	2.0800e−04	7.3790e−01	0.0000e+00	5.6843e−14	0.0000e+00	0.0000e+00
	Rank	6	8	7	5	9	**1**	4	**1**	**1**
F7	mean	7.1873e+02	7.2008e+02	7.1893e+02	7.2085e+02	7.3306e+02	7.1808e+02	7.2227e+02	7.2168e+02	**7.1592e+02**
	std	6.5319e+00	7.1092e+00	3.4893e+00	9.1208e−01	3.3733e+00	3.2000e+00	4.3854e+00	2.5130e+00	2.9380e+00
	Rank	3	5	4	6	9	2	8	7	**1**
F8	mean	8.0555e+02	8.2365e+02	8.0721e+02	8.1069e+02	8.1362e+02	8.0670e+02	8.0965e+02	8.0882e+02	**8.0547e+02**
	std	1.4945e+00	2.9050e+00	6.6682e+00	2.7399e+00	3.6753e+00	3.4500e+00	5.7626e+00	4.9970e+00	1.9052e+00
	Rank	2	9	4	7	8	3	6	5	**1**
F9	mean	9.0000e+02	9.0002e+02	9.0239e+02	9.0000e+02	9.1371e+02	**9.0000e+02**	**9.0000e+02**	**9.0000e+02**	**9.0000e+02**
	std	1.1774e−09	4.4764e−02	2.5090e+00	4.4100e−08	1.3749e+01	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00
	Rank	5	7	8	6	9	**1**	**1**	**1**	**1**
F10	mean	1.3902e+03	1.6931e+03	1.5922e+03	1.4880e+03	1.4896e+03	1.3442e+03	1.3935e+03	1.6172e+03	**1.1871e+03**
	std	4.7846e+02	2.1309e+02	2.5953e+02	1.6898e+02	1.3492e+02	1.6621e+02	3.3736e+02	1.5384e+02	1.2606e+02
	Rank	2	9	7	5	6	3	4	8	**1**
F11	mean	1.1023e+03	1.1207e+03	1.1226e+03	1.1041e+03	1.1281e+03	1.1047e+03	1.1025e+03	1.1027e+03	**1.1019e+03**
	std	8.7930e−01	1.0524e+01	1.3505e+01	1.7878e+00	5.4788e+00	1.8400e+00	1.2327e+00	1.3635e+00	1.4227e+00
	Rank	2	7	8	5	9	6	3	4	**1**
F12	mean	2.4939e+04	9.5954e+03	1.6029e+03	**1.2737e+03**	6.6483e+05	1.2529e+04	5.2558e+04	5.1211e+04	1.5602e+03
	std	1.5630e+04	5.7465e+03	2.2239e+02	7.7993e+01	2.8612e+05	6.9199e+03	4.1551e+04	5.8229e+04	1.9093e+02
	Rank	6	4	3	**1**	9	5	8	7	2
F13	mean	1.6183e+03	2.4985e+03	1.3606e+03	1.3133e+03	9.9402e+03	3.3969e+03	1.8664e+03	1.7454e+03	**1.3091e+03**
	std	1.6342e+02	5.9892e+02	4.6834e+01	3.2516e+00	1.2557e+04	2.6185e+03	4.4649e+02	9.0243e+02	1.2573e+00
	Rank	5	7	3	2	9	8	6	4	**1**
F14	mean	1.4439e+03	1.4598e+03	1.4465e+03	1.4037e+03	1.8197e+03	1.8192e+03	1.4345e+03	1.4434e+03	**1.4018e+03**
	std	1.2273e+01	2.9703e+01	6.0949e+01	2.2062e+00	1.7825e+02	7.0696e+02	4.5808e+01	5.7142e+01	1.0865e+00
	Rank	5	7	6	2	9	8	3	4	**1**
F15	mean	1.5197e+03	1.6367e+03	1.5203e+03	**1.5015e+03**	3.4885e+03	1.7182e+03	1.5347e+03	1.5165e+03	1.5019e+03
	std	1.4563e+01	7.0546e+01	3.2323e+01	4.6569e−01	8.4145e+02	2.2507e+02	3.2011e+01	1.0995e+01	1.0094e+00
	Rank	4	7	5	**1**	9	8	6	3	2
F16	mean	1.6024e+03	1.6815e+03	1.6549e+03	1.6012e+03	1.6093e+03	1.6251e+03	1.6014e+03	1.6013e+03	**1.6007e+03**
	std	4.9562e−01	1.1501e+02	7.6684e+01	2.1228e−01	6.0005e+01	4.8540e+01	6.9094e−01	7.4807e−01	1.8907e−01
	Rank	5	9	8	2	6	7	4	3	**1**
F17	mean	1.7287e+03	1.7710e+03	1.7395e+03	1.7139e+03	1.7440e+03	1.7101e+03	1.7025e+03	1.7052e+03	**1.7017e+03**
	std	1.3644e+01	5.2007e+01	4.6139e+00	8.9295e+00	1.4004e+01	9.4100e+00	3.3068e+00	8.5146e+00	2.0526e+00
	Rank	6	9	7	5	8	4	2	3	**1**
F18	mean	6.0656e+03	1.6945e+04	1.8451e+03	1.8114e+03	4.5009e+04	3.2778e+03	3.4646e+03	3.5255e+03	**1.8073e+03**
	std	2.8692e+03	1.2271e+04	2.3952e+01	7.3747e+00	1.5409e+04	1.0879e+03	1.2003e+03	1.1858e+03	1.0257e+01
	Rank	7	8	3	2	9	4	5	6	**1**
F19	mean	1.9241e+03	4.4732e+03	1.9179e+03	1.9008e+03	3.0928e+03	3.1458e+03	1.9297e+03	1.9353e+03	**1.9006e+03**
	std	7.7170e+00	3.5478e+03	1.4835e+01	4.6652e−01	4.1377e+02	1.3098e+03	2.1849e+01	5.0681e+01	5.8315e−01
	Rank	4	9	3	2	7	8	5	6	**1**
F20	mean	2.0152e+03	2.0444e+03	2.0188e+03	2.0262e+03	2.0426e+03	2.0034e+03	**2.0000e+03**	2.0000e+03	**2.0000e+03**
	std	1.1440e+01	2.7244e+01	7.2757e+00	1.2011e+01	5.1477e+00	4.9200e+00	0.0000e+00	9.6085e−02	0.0000e+00
	Rank	5	9	6	7	8	4	**1**	3	**1**
F21	mean	2.2528e+03	2.3221e+03	2.2013e+03	2.2000e+03	2.3220e+03	2.2294e+03	2.2310e+03	2.2165e+03	**2.2000e+03**
	std	6.0960e+01	5.2210e+00	1.5169e+00	1.4100e−05	5.8569e+01	4.8790e+01	5.2016e+01	3.3746e+01	2.2737e−12
	Rank	9	6	3	2	5	7	8	4	**1**
F22	mean	2.3051e+03	2.2807e+03	2.3017e+03	2.2289e+03	**2.2275e+03**	2.2888e+03	2.2903e+03	2.3005e+03	2.2698e+03
	std	5.0892e−01	4.1771e+01	8.2982e−01	4.8177e+01	4.4290e+01	2.8460e+01	2.2567e+01	5.5010e−01	3.9118e+01
	Rank	9	4	8	2	**1**	5	6	7	3
F23	mean	2.6077e+03	2.6248e+03	**2.5360e+03**	2.6084e+03	2.6167e+03	2.6080e+03	2.6108e+03	2.6144e+03	2.6086e+03
	std	1.6465e+00	7.8870e+00	1.5741e+02	1.5922e+00	5.9111e+00	3.6800e+00	4.0616e+00	4.7848e+00	4.5021e+00
	Rank	2	9	**1**	4	8	3	6	7	5

(*continued on next page*)

Table 2 (*continued*).

F24	mean	2.7331e+03	2.7575e+03	2.5622e+03	2.5000e+03	2.5197e+03	2.5780e+03	2.6140e+03	2.5984e+03	**2.5000e+03**
	std	2.0963e+00	8.8522e+00	1.2438e+02	2.0940e−04	1.1862e+02	1.1188e+02	9.9057e+01	9.4793e+01	3.2260e−12
	Rank	5	6	6	2	2	3	4	7	**1**
F25	mean	2.8985e+03	2.9450e+03	2.9341e+03	**2.8977e+03**	2.9464e+03	2.9092e+03	2.9077e+03	2.8990e+03	2.8989e+03
	std	7.7252e−01	2.5352e+01	2.3671e+01	4.7300e−08	2.0305e+01	1.9400e+01	2.0375e+01	8.6919e−01	8.1733e−01
	Rank	2	8	7	**1**	9	6	5	4	3
F26	mean	2.9000e+03	3.0105e+03	2.9363e+03	2.8000e+03	2.9304e+03	2.8367e+03	2.8816e+03	2.9024e+07	**2.7500e+03**
	std	8.8640e−08	1.5323e+01	4.6713e+01	1.4142e+02	1.7213e+01	8.0870e+01	4.1105e+01	2.1252e+01	1.0000e+02
	Rank	5	9	8	2	7	3	4	6	**1**
F27	mean	**3.0893e+03**	3.0978e+03	3.0923e+03	3.0897e+03	3.0922e+03	3.0933e+03	3.0923e+03	3.0912e+03	3.0904e+03
	std	2.8359e−01	2.2075e+00	2.0021e+00	4.8343e−01	1.2429e+00	4.6200e+00	2.9884e+00	1.9371e+00	8.7410e−01
	Rank	**1**	9	6	2	5	8	7	4	3
F28	mean	3.3198e+03	3.3631e+03	3.1586e+03	3.1000e+03	3.1891e+03	3.1049e+03	3.1544e+03	3.1531e+03	**3.1000e+03**
	std	1.4714e+02	9.7751e+01	6.7640e+01	4.9900e−05	1.3101e+02	1.0462e+02	3.0519e+01	3.7539e+01	5.0502e−12
	Rank	8	9	6	2	7	3	5	4	**1**
F29	mean	3.1464e+03	3.2038e+03	3.1431e+03	**3.1418e+03**	3.1956e+03	3.1648e+03	3.1805e+03	3.1768e+03	3.1476e+03
	std	9.7921e+00	9.6117e+01	1.4434e+01	5.2609e+00	8.3746e+00	1.8960e+01	1.9516e+01	1.3608e+01	5.2967e+00
	Rank	3	9	2	**1**	8	5	7	6	4
F30	mean	5.9412e+03	4.1697e+05	8.3190e+05	3.5948e+03	2.6601e+05	1.2169e+04	2.7998e+04	3.4936e+04	**3.5672e+03**
	std	1.3336e+03	4.6605e+05	5.8848e+05	2.4351e+02	1.2519e+05	8.6672e+03	3.2509e+04	2.8056e+04	8.5766e+01
	Rank	3	8	9	2	7	4	5	6	**1**
Count		2	0	2	4	1	2	2	2	**20**
Ave.Rank		4.55	7.59	5.21	3.24	7.41	4.59	5.03	4.97	**1.62**
Total.Rank		3	9	7	2	8	4	6	5	**1**

both R^+ and R^- values. The determination of the p-value, through the calculation of R^+ and R^-, serves as a critical indicator of statistical significance. A p-value less than 0.05 strongly suggests a significant performance disparity between RLHBO and the algorithm it is compared against. The notation $n/w/t/l$ provides a comprehensive summary: 'n' stands for the total number of functions assessed, 'w' indicates the number of functions where RLHBO outshines the comparative algorithm, 't' denotes the functions with comparable performance between RLHBO and the comparison algorithm, and 'l' signifies the functions where RLHBO's performance is not as strong. This categorization presents a nuanced understanding of RLHBO's comparative advantage, supporting a detailed and statistically sound evaluation of its performance across various dimensions.

Analyzing the results from Table 6, Table 7, and Table 8, it becomes clear that all p-values listed are below the threshold of 0.05. This statistically significant finding underscores the substantial difference in performance between RLHBO and all the comparative algorithms. Specifically, in the context of the 10-dimensional function within the CEC2017 benchmarks, RLHBO not only outshines HBO in 27 out of 29 functions but also matches its performance in 2 functions. This trend of outperformance continues in the 30-dimensional function, where RLHBO surpasses HBO in 28 functions and equals in performance in 1 function. The 50-dimensional function further attests to RLHBO's superiority, with RLHBO outperforming HBO in 29 functions.

These results vividly illustrate that RLHBO, as introduced in this paper, significantly excels beyond HBO, affirming that the strategies implemented for enhancing RLHBO's performance are indeed effective. The p-value is less than 0.05 in comparisons with not just HBO but also MPA, IGWO, COA, HODEI, CGO, MPSO, and BWO further cements RLHBO's position as the leading algorithm. In essence, RLHBO markedly outperforms all other algorithms under review across the 10, 30, and 50-dimensional functions of the CEC2017.

In summary, RLHBO showcases unparalleled optimization capabilities over HBO and other benchmarked algorithms, thereby validating the effectiveness of the methodologies proposed in this paper to boost RLHBO's optimization performance.

4.5. Effect of each improvement on RLHBO

To rigorously assess the impact of the three innovative strategies introduced in this study, we designed comparative experiments featuring the newly proposed RLHBO algorithm against its four derivatives – HBO1, HBO2, HBO3, and HBO1+2 – as well as the baseline HBO algorithm. Each variant, HBO1, HBO2, and HBO3, is engineered to incorporate one of the strategies proposed in this paper. Specifically: HBO1 adopts an accelerated convergence strategy, incorporating a dimension self-learning approach and a convergence strategy based on search direction similarity, building on the HBO framework. HBO2 introduces a dynamic control search strategy utilizing reinforcement learning to HBO, while HBO3 applies adversarial learning for initializing the HBO population. HBO1+2 combines the strategies of accelerated convergence and dynamic control search using reinforcement learning.

To ensure fairness, we set the experimental parameters of all algorithms to be the same, and conduct tests using a 10-dimensional CEC2017 function. The test results are shown in the Table 9: + indicates better performance than the comparison algorithms; ≈ indicates that the method obtains approximate optimization results; − indicates worse performance than the comparison algorithms; The last row presents the statistical results as $n/+/\approx/-$.

The analysis of the data from Table 9 underscores the superior performance of the RLHBO algorithm and its four derivative variants over the original HBO algorithm, affirming the potency of the strategies introduced in this paper. HBO1 showcases a notable advancement, besting HBO in 22 functions while underperforming in 5. This robust performance validates the dimension of self-learning and convergence strategy based on search direction similarity, illustrating their pivotal role in refining the algorithm's efficiency. HBO2 records an impressive stride forward in 22 functions as well, though it does not fare as well in 6 functions. This underscores the success of the dynamic control search strategy empowered by reinforcement learning, indicating its significant contribution to augmenting HBO's optimization capacity. HBO3 also registers a commendable performance, outperforming HBO in 17 functions and lagging in 10. This outcome suggests that the quasi-oppositional learning strategy for initialization effectively enhances the algorithm's performance, providing a stronger foundation for optimization. HBO1+2, amalgamating the accelerated convergence and dynamic control search strategy utilizing reinforcement learning, excels in 26 functions and falls short in only one, highlighting the synergistic effect of combining these strategies to push the boundaries of optimization performance. The RLHBO algorithm, incorporating all three proposed strategies, demonstrates unmatched superiority across the 29 functions tested. It equals HBO's performance in just two functions but surpasses it in the remaining 27, conclusively proving the effectiveness of the introduced strategies in enhancing the HBO algorithm. This comprehensive performance advantage underscores each strategy's critical contribution to the RLHBO algorithm's success.

Table 3
Optimization results and comparison for CEC-2017 test functions(the dimension m = 30).

Function	Metric	IGWO	MPSO	CGO	MPA	BWO	COA	HBO	HODEI	RLHBO
F1	mean	4.3335e+03	8.0039e+09	1.2692e+02	1.6650e+03	4.6104e+10	1.3192e+04	1.6377e+02	1.4913e+02	**1.0000e+02**
	std	3.4253e+03	5.4960e+09	1.0363e+01	1.4490e+03	2.6252e+09	2.0128e+04	7.4203e+01	5.2809e+01	7.8713e−11
	Rank	6	8	2	5	9	7	4	3	**1**
F3	mean	1.0144e+03	2.6862e+04	3.0000e+02	3.0002e+02	6.4201e+04	1.1551e+03	4.4793e+03	1.4292e+03	**3.0000e+02**
	std	1.2102e+03	3.2812e+04	8.9073e−10	1.8700e−02	1.4566e+03	2.4226e+02	3.0001e+03	6.3071e+02	1.8418e−12
	Rank	4	8	2	3	9	5	7	6	**1**
F4	mean	4.9459e+02	1.3234e+03	4.1564e+02	4.8589e+02	9.8641e+03	4.9418e+02	4.8876e+02	4.9908e+02	**4.0444e+02**
	std	1.5212e+01	4.8200e+02	2.8676e+01	4.8820e−01	4.1264e+02	1.3364e+01	2.2513e+00	1.2981e+01	7.5640e−01
	Rank	5	8	2	3	9	6	4	7	**1**
F5	mean	**5.4231e+02**	6.3634e+02	5.7711e+02	5.8857e+02	8.8113e+02	6.3276e+02	6.2116e+02	6.3105e+02	5.4855e+02
	std	5.4559e+00	2.1977e+01	1.4176e+01	1.3848e+01	1.5203e+01	2.7752e+01	8.8456e+00	1.1954e+01	9.2406e+00
	Rank	**1**	8	3	4	9	6	5	7	2
F6	mean	6.0000e+02	6.1543e+02	6.2162e+02	6.0117e+02	6.7888e+02	6.0348e+02	**6.0000e+02**	**6.0000e+02**	**6.0000e+02**
	std	6.5637e−12	5.6730e+00	2.5422e+00	4.4040e−01	5.2168e+00	1.2371e+00	0.0000e+00	0.0000e+00	0.0000e+00
	Rank	4	7	8	5	9	6	**1**	**1**	**1**
F7	mean	7.9013e+02	9.3792e+02	8.4754e+02	8.0138e+02	1.3440e+03	8.3234e+02	8.6817e+02	8.5717e+02	**7.8545e+02**
	std	6.0970e+01	9.5267e+01	3.2904e+01	1.6290e+01	1.2076e+01	2.5872e+01	8.4131e+00	1.3126e+01	4.9967e+00
	Rank	2	8	5	3	9	4	7	6	**1**
F8	mean	**8.3123e+02**	9.4575e+02	8.7288e+02	8.7612e+02	1.1167e+03	8.8132e+02	9.2833e+02	9.3939e+02	8.6332e+02
	std	8.8744e+00	1.9537e+01	1.1471e+01	5.8805e+00	4.1079e+00	1.0317e+01	7.7360e+00	1.4813e+01	1.6051e+01
	Rank	**1**	8	3	4	9	5	6	7	2
F9	mean	**9.0000e+02**	3.7265e+03	1.5544e+03	9.3534e+02	8.8487e+03	1.0552e+03	9.0106e+02	9.0124e+02	9.0095e+02
	std	0.0000e+00	1.6701e+03	3.2432e+02	2.8329e+01	1.6159e+03	1.0881e+02	1.1132e+00	9.2920e−01	1.0815e+00
	Rank	**1**	8	7	5	9	6	3	4	2
F10	mean	5.1526e+03	5.7459e+03	4.6652e+03	3.8052e+03	8.2398e+03	4.0330e+03	5.4719e+03	5.5043e+03	**3.4114e+03**
	std	3.1421e+03	7.3229e+02	9.7596e+02	1.0836e+02	3.0296e+02	3.7671e+02	2.7091e+02	1.5198e+02	3.0557e+02
	Rank	5	8	4	2	9	3	6	7	**1**
F11	mean	1.1427e+03	3.7955e+03	1.2222e+03	1.1220e+03	5.4825e+03	1.1744e+03	1.1546e+03	1.1403e+03	**1.1170e+03**
	std	3.3745e+01	4.2167e+03	3.1327e+01	3.7300e+00	1.4371e+03	3.7468e+01	3.4825e+01	3.1823e+01	3.0180e+01
	Rank	5	8	7	2	9	6	3	4	**1**
F12	mean	5.7822e+05	7.4800e+08	1.5190e+04	**2.8922e+03**	8.1284e+09	2.5289e+04	1.9206e+06	1.6862e+06	9.1453e+03
	std	3.4034e+05	6.9358e+08	1.0936e+04	1.5122e+02	1.8488e+09	5.9512e+03	1.4549e+06	7.7978e+05	5.9891e+03
	Rank	5	8	3	**1**	9	4	7	6	2
F13	mean	3.1741e+04	1.9453e+07	1.2694e+04	1.4051e+03	3.2025e+09	1.9291e+03	1.4089e+04	1.2865e+04	**1.3837e+03**
	std	2.0704e+04	3.4978e+07	9.9825e+03	1.3752e+01	2.0036e+09	3.9275e+02	1.2203e+04	6.0815e+03	5.2158e+01
	Rank	7	8	4	2	9	3	6	5	**1**
F14	mean	3.0391e+03	1.8205e+05	1.5798e+03	**1.4304e+03**	4.8067e+05	1.4416e+03	1.9886e+04	1.8081e+04	1.4472e+03
	std	6.4249e+02	1.7479e+05	7.5705e+01	5.1018e+00	3.0228e+05	3.9846e+00	1.5360e+04	1.2273e+04	7.6275e+00
	Rank	5	8	4	**1**	9	2	7	6	3
F15	mean	1.2705e+04	1.0184e+05	9.2545e+03	1.5234e+03	1.2158e+08	1.6267e+03	2.8166e+03	2.4967e+03	**1.5192e+03**
	std	5.4118e+03	4.6564e+05	1.4297e+04	3.2559e+00	5.4875e+07	2.6879e+01	1.1424e+03	8.6220e+02	6.3063e+00
	Rank	7	8	6	2	9	3	5	4	**1**
F16	mean	**1.7537e+03**	2.9329e+03	2.2633e+03	2.1289e+03	4.6792e+03	2.3474e+03	2.3616e+03	2.4292e+03	1.9868e+03
	std	9.6657e+01	3.2295e+02	2.6665e+02	1.0486e+02	1.4374e+02	2.3917e+02	1.6060e+02	1.8130e+02	5.9719e+01
	Rank	**1**	8	5	3	9	4	6	7	2
F17	mean	1.8283e+03	2.3314e+03	2.2087e+03	1.7770e+03	3.4983e+03	1.8538e+03	1.8644e+03	1.8664e+03	**1.7600e+03**
	std	9.1511e+01	2.4127e+02	2.3365e+02	1.3245e+01	2.3290e+02	8.0492e+01	8.3386e+01	8.9809e+01	2.5296e+01
	Rank	3	8	7	2	9	4	5	6	**1**
F18	mean	4.9344e+04	9.5907e+05	3.7860e+03	**1.8303e+03**	1.3575e+07	1.9017e+03	4.8811e+05	7.8227e+05	2.6516e+03
	std	2.2602e+04	8.6867e+05	2.4498e+03	4.9801e+00	5.8592e+06	1.7295e+01	3.1722e+04	9.3270e+04	6.4857e+02
	Rank	5	8	4	**1**	9	2	6	7	3
F19	mean	1.0614e+04	7.2822e+05	2.1363e+03	1.9232e+03	1.3679e+08	1.9247e+03	5.6508e+03	5.2716e+03	**1.9165e+03**
	std	4.2874e+03	1.1845e+05	7.0044e+01	2.6105e+00	5.7490e+07	3.6924e+00	2.8614e+03	2.0534e+03	7.0787e+00
	Rank	7	8	4	2	9	3	6	5	**1**
F20	mean	2.1295e+03	2.3340e+03	2.3414e+03	**2.1159e+03**	2.7412e+03	2.1850e+03	2.1884e+03	2.1732e+03	2.1655e+03
	std	6.5639e+01	1.7230e+02	2.4564e+02	6.3998e+01	1.1978e+02	8.8641e+01	8.8165e+01	9.3379e+01	6.3889e+01
	Rank	2	7	8	**1**	9	5	6	4	3
F21	mean	2.3317e+03	2.4479e+03	2.3790e+03	2.2965e+03	2.6645e+03	2.3504e+03	2.4264e+03	2.4373e+03	**2.2602e+03**
	std	4.7774e+00	2.6579e+01	1.6702e+01	8.8117e+01	3.5338e+01	2.8274e+01	1.8584e+01	1.3746e+01	8.3295e+01
	Rank	3	9	4	2	8	7	5	6	**1**
F22	mean	3.5242e+03	6.3504e+03	2.3020e+03	2.3004e+03	7.3749e+03	2.3031e+03	3.0790e+03	2.3005e+03	**2.3000e+03**
	std	1.3651e+03	1.1202e+03	2.3115e+00	3.6800e−02	9.2909e+02	2.6416e+03	2.1229e+03	1.4164e+00	6.9464e−13
	Rank	7	8	4	2	9	5	6	3	**1**
F23	mean	2.6669e+03	2.9195e+03	2.8017e+03	2.7074e+03	3.2576e+03	**2.6416e+03**	2.7675e+03	2.7834e+03	2.6844e+03
	std	9.5416e+00	4.6202e+01	1.0779e+01	2.3656e+01	2.1668e+01	1.2352e+02	1.1371e+01	1.3234e+01	1.1455e+01
	Rank	2	8	7	4	9	**1**	5	6	3

(*continued on next page*)

Table 3 (*continued*).

F24	mean	**2.8504e+03**	3.0764e+03	2.9172e+03	2.8712e+03	3.4472e+03	2.8790e+03	3.0139e+03	3.0160e+03	2.8849e+03
	std	1.4433e+01	4.7166e+01	3.7064e+01	6.3414e+00	2.9568e+01	1.6475e+01	7.9690e+00	1.2765e+01	2.1754e+01
	Rank	**1**	8	5	2	9	3	6	7	4
F25	mean	2.8865e+03	3.4563e+03	2.9011e+03	2.8862e+03	4.1938e+03	3.1199e+03	2.8866e+03	2.8871e+03	**2.8854e+03**
	std	2.1457e+00	5.3208e+02	2.2588e+01	1.8124e−01	1.7029e+02	1.0849e+02	2.2290e−01	4.5360e−01	2.0560e+00
	Rank	3	8	6	2	9	7	4	5	**1**
F26	mean	3.7771e+03	4.2208e+03	5.1390e+03	2.9000e+03	9.5731e+03	2.9004e+03	4.7390e+03	4.9277e+03	**2.8485e+03**
	std	1.4519e+02	2.7003e+02	1.3555e+03	3.8600e−02	6.0030e+02	1.8123e−01	1.5277e+02	8.3446e+01	4.9587e+01
	Rank	4	8	5	2	9	3	6	7	**1**
F27	mean	**3.1915e+03**	3.2534e+03	3.2661e+03	3.2081e+03	3.7296e+03	3.4129e+03	3.2100e+03	3.2095e+03	3.2084e+03
	std	1.2689e+01	4.7692e+01	4.1195e+01	5.1673e+00	9.3177e+01	5.3559e+01	8.6856e+00	9.0779e+00	2.8512e+00
	Rank	**1**	7	6	2	9	8	5	4	3
F28	mean	3.1428e+03	3.7605e+03	3.1329e+03	3.1253e+03	5.9518e+03	3.2188e+03	3.1444e+03	3.1216e+03	**3.1000e+03**
	std	3.2316e+01	4.7857e+02	7.1859e+01	4.7779e+01	2.1632e+02	2.5178e+01	6.1794e+01	4.8726e+01	2.6766e−07
	Rank	5	8	4	3	9	7	6	2	**1**
F29	mean	3.4310e+03	4.0583e+03	4.2088e+03	3.4348e+03	5.8118e+03	3.6372e+03	3.6459e+03	3.6902e+03	**3.4047e+03**
	std	5.0197e+01	3.0716e+02	3.6661e+02	1.1135e+02	4.3898e+02	9.7555e+01	1.0434e+02	1.0779e+02	2.9294e+01
	Rank	2	7	8	3	9	4	6	5	**1**
F30	mean	2.6770e+04	6.6055e+06	5.8258e+03	**5.2707e+03**	4.0378e+08	7.5242e+03	2.7782e+04	2.7053e+04	6.1930e+03
	std	9.5072e+03	9.2792e+06	5.2523e+02	1.4025e+02	1.5310e+08	1.2924e+03	1.8114e+04	1.2100e+04	2.2576e+02
	Rank	5	8	2	**1**	9	4	7	6	3
Count		6	0	0	5	0	1	1	1	**17**
Ave.Rank		3.76	7.90	4.79	2.55	8.97	4.48	5.38	5.28	**1.69**
Total.Rank		3	8	4	2	9	5	7	6	**1**

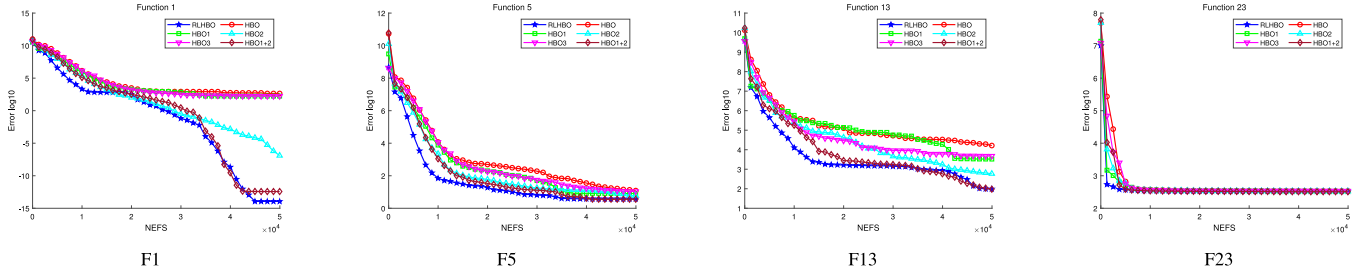


Fig. 5. Convergence curves of HBO with the proposed strategy versus HBO on CEC2017 in 10 dimensions.

These findings conclusively demonstrate that each proposed strategy significantly boosts the HBO algorithm's optimization capability. By integrating these strategies, the RLHBO algorithm not only sets a new benchmark for optimization performance but also solidifies the indispensability of each strategy for achieving superior optimization outcomes.

The detailed analysis of convergence curves for four selected functions from the 10-dimensional CEC-2017 benchmark, as depicted in Fig. 5, illustrates the superior performance of HBO variants employing the strategies introduced in this study over the standard HBO algorithm. This visual comparison clearly highlights the efficacy of the proposed enhancements. The HBO1+2 and RLHBO algorithms, which incorporate two and three of the proposed strategies respectively, demonstrate the most impressive performance. Their ability to outpace the standard HBO in terms of both speed and efficiency in convergence is evident, showcasing the synergistic effect of combining multiple optimization strategies. Among the variants utilizing a single strategy, HBO2 stands out significantly. The inclusion of a dynamic control strategy powered by reinforcement learning offers HBO a substantial performance uplift. This suggests that the adaptive adjustments to the search process, guided by reinforcement learning principles, are crucial for overcoming optimization challenges. Notably, HBO1+2 achieves a faster convergence rate compared to HBO2 in the later stages of optimization. This observation underscores the impact of the accelerated convergence strategy, highlighting its role in enhancing the convergence speed and ensuring stability in the search process.

The RLHBO algorithm, which amalgamates all three proposed strategies, outshines HBO1+2 in performance. This superiority indicates that the quasi-oppositional learning strategy, aimed at optimizing population initialization, further augments the algorithm's effectiveness. Such an approach not only accelerates the initial search phase but also contributes to a more diverse and robust exploration of the solution space.

These findings collectively affirm the proposed strategies' pivotal role in elevating HBO's optimization capabilities. The accelerated convergence, dynamic control through reinforcement learning, and quasi-oppositional learning for initialization each play a distinct part in improving the algorithm's performance. Moreover, the comprehensive enhancement observed with the RLHBO algorithm, which integrates all three strategies, further validates their effectiveness and underscores the benefits of their combined application in complex optimization scenarios.

4.6. Stability analysis

The stability of the RLHBO algorithm, a crucial aspect of its overall performance, was rigorously assessed through experiments utilizing the 30-dimensional test functions of the CEC2017 suite. This evaluation selected a diverse range of functions encompassing unimodal, multimodal, hybrid, and composite types to offer a comprehensive insight into RLHBO's robustness across different problem landscapes. This paper utilizes boxplots to analyze and compare the stability of algorithms. The center mark of the boxplot represents the median value of the 31 times the algorithm ran the function, and the symbol + represents outliers.

It can be observed from function f_1 in Fig. 6 that RLHBO, IGWO, and CGO exhibit the best stability, with RLHBO demonstrating the highest convergence accuracy. For function f_6, RLHBO, HBO, and HODEI display the best convergence accuracy and stability. In function f_{20}, HBO contains outliers, while RLHBO shows no outliers and demonstrates the best stability. In function f_{26}, RLHBO and MPA are the only

Table 4
Optimization results and comparison for CEC-2017 test functions(the dimension m = 50).

Function	Metric	IGWO	MPSO	CGO	MPA	BWO	COA	HBO	HODEI	RLHBO
F1	mean	1.1524e+04	3.6883e+10	3.6323e+03	6.8966e+03	9.3970e+10	3.3153e+06	2.8614e+03	1.3306e+03	**1.0000e+02**
	std	1.2687e+04	8.4531e+09	1.9952e+03	1.6194e+03	9.4960e+09	1.0758e+06	3.1778e+03	1.2709e+03	6.2702e−06
	Rank	6	8	4	5	9	7	3	2	**1**
F3	mean	8.3850e+03	5.1774e+04	3.0000e+02	3.0793e+02	1.5686e+05	1.9036e+04	4.0851e+04	3.3398e+04	**3.0000e+02**
	std	3.8599e+03	1.7767e+04	3.2576e−04	4.4108e+00	1.6962e+04	2.7062e+03	9.3231e+03	8.5316e+03	1.6179e−05
	Rank	4	8	2	3	9	5	7	6	**1**
F4	mean	4.6857e+02	4.0370e+03	4.7404e+02	5.1800e+02	2.7298e+04	5.2087e+02	4.3557e+02	4.4825e+02	**4.3056e+02**
	std	6.1230e+01	4.7335e+02	4.9356e+01	4.3238e+01	1.4040e+03	2.1646e+01	1.2893e+01	3.1587e+01	3.3221e+00
	Rank	4	8	5	7	9	6	2	3	**1**
F5	mean	**5.6816e+02**	8.5248e+02	7.1143e+02	6.4303e+02	1.1338e+03	7.2356e+02	8.4168e+02	7.9409e+02	6.0141e+02
	std	1.8533e+01	6.4649e+01	3.1426e+01	2.4873e+01	2.1869e+01	5.1098e+01	8.5545e+00	2.7244e+01	3.3469e+01
	Rank	**1**	8	4	3	9	5	7	6	2
F6	mean	6.0003e+02	6.2893e+02	6.2959e+02	6.0257e+02	6.9753e+02	6.1200e+02	6.0000e+02	6.0000e+02	**6.0000e+02**
	std	5.0100e−02	1.0213e+01	7.4835e+00	6.7050e−01	3.6314e+00	2.9277e+00	8.1865e−07	2.6541e−08	2.3971e−08
	Rank	4	7	8	5	9	6	3	2	**1**
F7	mean	8.3068e+02	1.3168e+03	1.1081e+03	9.4862e+02	1.8511e+03	1.0378e+03	1.0855e+03	1.0565e+03	**8.2992e+02**
	std	1.5713e+01	1.6767e+02	5.2931e+01	2.9874e+01	3.6741e+01	4.8529e+01	1.2490e+01	"14.5969e+00"	1.7818e+01
	Rank	2	8	7	3	9	4	6	5	**1**
F8	mean	**8.9630e+02**	1.2126e+03	9.7362e+02	9.5397e+02	1.4528e+03	9.9912e+02	1.1376e+03	1.1027e+03	9.1101e+02
	std	3.0473e+01	1.0264e+02	2.6418e+01	3.1185e+01	2.7129e+01	1.2788e+01	1.4991e+01	2.4112e+01	2.2999e+01
	Rank	**1**	8	4	3	9	5	7	6	2
F9	mean	**9.0039e+02**	8.2298e+03	4.5926e+03	1.3554e+03	3.3153e+04	3.8071e+03	9.2873e+02	9.2732e+02	9.1808e+02
	std	2.6090e−01	2.7889e+03	7.9914e+02	1.0311e+02	1.4002e+03	1.4098e+03	1.2475e+01	3.4529e+01	1.4132e+01
	Rank	**1**	8	7	5	9	6	4	3	2
F10	mean	6.8871e+03	8.3873e+03	7.2515e+03	**5.8017e+03**	1.4003e+04	6.4745e+03	1.0182e+04	9.7222e+03	6.7547e+03
	std	4.4226e+03	7.1587e+02	1.5504e+02	4.0396e+02	4.9581e+02	6.4291e+02	3.9641e+02	4.7790e+02	8.2407e+02
	Rank	4	6	5	**1**	9	2	8	7	3
F11	mean	**1.1507e+03**	4.2200e+03	1.2953e+03	1.2441e+03	1.8174e+04	1.2631e+03	1.2078e+03	1.2099e+03	1.1878e+03
	std	7.1714e+00	1.4893e+03	2.5384e+01	2.8069e+01	1.3489e+03	3.7264e+01	7.0049e+01	3.7255e+01	2.1399e+01
	Rank	**1**	8	7	5	9	6	3	4	2
F12	mean	1.0592e+06	5.1850e+09	**2.8208e+04**	9.7388e+05	4.5561e+10	1.4000e+07	2.0820e+07	1.6651e+07	3.2869e+04
	std	2.5898e+05	2.8800e+09	1.3049e+04	1.0932e+05	4.0729e+09	1.0562e+06	6.9110e+06	6.0586e+06	2.3345e+04
	Rank	4	8	**1**	3	9	5	7	6	2
F13	mean	4.9697e+03	6.7469e+08	8.7510e+03	1.8733e+03	2.0040e+10	1.7285e+04	7.1710e+03	4.5334e+03	**1.4436e+03**
	std	1.7022e+03	7.2990e+08	7.1600e+03	2.2522e+02	9.1172e+09	5.4589e+03	5.9733e+03	3.2811e+03	3.3849e+01
	Rank	4	8	6	2	9	7	5	3	**1**
F14	mean	2.3469e+04	2.2132e+06	1.9532e+03	**1.4672e+03**	1.7969e+07	1.5714e+03	3.9939e+05	3.0621e+05	1.5832e+03
	std	1.3041e+04	2.4841e+06	1.2977e+02	5.4317e+00	6.3754e+06	1.8228e+01	2.3450e+05	1.8991e+05	1.0125e+02
	Rank	5	8	4	**1**	9	2	7	6	3
F15	mean	1.8221e+04	2.2716e+08	8.8976e+03	1.7274e+03	3.3986e+09	2.5384e+03	7.4871e+03	8.1082e+03	**1.5566e+03**
	std	1.1549e+04	2.2273e+08	7.6489e+03	4.6794e+01	7.1422e+08	2.4300e+02	6.9782e+03	7.0997e+03	1.1651e+01
	Rank	7	8	6	2	9	3	4	5	**1**
F16	mean	**2.0227e+03**	4.3248e+03	3.4416e+03	2.4353e+03	7.4207e+03	2.7762e+03	3.3246e+03	3.2504e+03	2.5869e+03
	std	2.3303e+02	3.0654e+02	3.8134e+02	1.4323e+02	4.9857e+02	1.7247e+02	1.0677e+02	1.2622e+02	1.7409e+02
	Rank	**1**	8	7	2	9	4	6	5	3
F17	mean	2.2789e+03	3.8877e+03	2.7998e+03	2.2980e+03	5.6626e+03	2.5936e+03	2.8210e+03	2.7305e+03	**2.1604e+03**
	std	2.9601e+02	4.1920e+02	3.9549e+02	2.1871e+02	6.2732e+02	4.8061e+01	1.2225e+02	1.9225e+02	1.4073e+02
	Rank	2	8	6	3	9	4	7	5	**1**
F18	mean	2.6746e+05	4.7459e+05	3.8353e+03	**1.9104e+03**	5.8828e+07	3.0549e+04	4.3629e+06	4.1513e+06	5.4849e+03
	std	1.3123e+05	2.4742e+05	5.3639e+02	5.2149e+01	2.5671e+07	1.8042e+04	2.0346e+06	2.2695e+06	2.4577e+03
	Rank	5	6	2	**1**	9	4	8	7	3
F19	mean	1.5715e+04	1.2969e+07	2.8384e+03	1.9456e+03	1.5221e+09	2.0922e+03	8.7118e+03	5.6479e+03	**1.9280e+03**
	std	1.6663e+04	1.9100e+07	8.1324e+02	1.0322e+01	7.3650e+08	4.4643e+01	5.9860e+03	2.3879e+03	6.0774e+00
	Rank	7	8	4	2	9	3	6	5	**1**
F20	mean	2.4307e+03	3.0336e+03	2.6829e+03	**2.3993e+03**	3.7187e+03	2.6154e+03	2.9574e+03	3.0012e+03	2.5745e+03
	std	3.2270e+02	5.4291e+02	1.6629e+02	3.5320e+01	1.3834e+02	2.1081e+02	6.9333e+01	4.7601e+01	2.1630e+02
	Rank	2	8	5	**1**	9	4	6	7	3
F21	mean	**2.3642e+03**	2.7084e+03	2.4900e+03	2.4063e+03	3.0864e+03	2.4583e+03	2.6260e+03	2.6290e+03	2.3972e+03
	std	1.6516e+01	5.2371e+01	3.4225e+01	2.2610e+01	3.4926e+01	2.5037e+01	5.7297e+00	2.8375e+01	1.3345e+01
	Rank	**1**	8	5	3	9	4	6	7	2
F22	mean	7.4958e+03	1.1111e+04	5.1742e+03	**2.3014e+03**	1.5929e+04	4.4412e+03	1.2020e+04	1.1349e+04	4.1963e+03
	std	4.8380e+03	6.6384e+02	3.3299e+03	1.7783e+00	1.6027e+02	2.8784e+03	7.3329e+02	3.0275e+02	3.7906e+03
	Rank	5	6	4	**1**	9	3	8	7	2
F23	mean	**2.7589e+03**	3.3832e+03	3.0563e+03	2.8028e+03	3.8352e+03	3.1573e+03	3.0684e+03	3.0543e+03	2.8654e+03
	std	9.2903e+00	2.0134e+01	3.3354e+01	2.2444e+01	4.0230e+01	1.0548e+02	1.8074e+01	2.2902e+01	3.6525e+01
	Rank	**1**	8	5	2	9	7	6	4	3

(*continued on next page*)

Table 4 (*continued*).

F24	mean	**2.9520e+03**	3.6314e+03	3.1314e+03	3.0008e+03	4.2842e+03	3.0754e+03	3.2750e+03	3.1944e+03	3.0118e+03
	std	1.5833e+01	1.6814e+02	5.4991e+01	1.6605e+01	5.6813e+01	3.0280e+01	1.0366e+01	1.3536e+02	4.3987e+01
	Rank	**1**	8	5	2	9	4	7	6	3
F25	mean	3.0548e+03	5.4069e+03	3.0498e+03	3.0206e+03	1.2220e+04	3.0537e+03	3.0238e+03	3.0361e+03	**3.0157e+03**
	std	2.1894e+01	1.4233e+03	2.0863e+01	4.0617e+01	8.2601e+02	2.2909e+01	2.3957e+01	1.9590e+01	3.2211e+01
	Rank	7	8	5	2	9	6	3	4	**1**
F26	mean	4.3018e+03	1.0164e+04	6.9246e+03	**3.4396e+03**	1.5958e+04	3.6254e+03	7.0329e+03	6.7063e+03	4.0396e+03
	std	2.4643e+02	1.1942e+03	8.7799e+02	1.0791e+03	2.7768e+02	2.4950e+02	1.8634e+02	1.8416e+02	1.1473e+03
	Rank	3	8	6	**1**	9	2	7	5	4
F27	mean	3.2612e+03	3.8273e+03	3.8741e+03	3.2826e+03	5.1804e+03	3.3285e+03	3.2549e+03	3.2598e+03	**3.2471e+03**
	std	3.7876e+01	1.8597e+02	1.7361e+02	5.3680e+01	2.5396e+02	3.7020e+01	5.1280e+00	2.4928e+01	2.3560e+01
	Rank	4	7	8	5	9	6	2	3	**1**
F28	mean	3.2782e+03	9.6695e+03	3.2992e+03	3.2990e+03	1.1013e+04	3.5952e+03	3.2777e+03	3.2732e+03	**3.2724e+03**
	std	2.2300e+01	6.9795e+02	3.0985e+01	2.1143e+01	3.8273e+02	2.8943e+01	2.2265e+01	1.8142e+01	1.7190e+01
	Rank	4	8	6	5	9	7	3	2	**1**
F29	mean	**3.4877e+03**	5.6503e+03	4.5585e+03	3.6705e+03	1.1519e+04	4.1610e+03	4.1339e+03	4.1649e+03	3.6228e+03
	std	1.2484e+02	3.2240e+02	7.0315e+02	1.6028e+02	6.2332e+02	2.9342e+02	9.1599e+01	7.6456e+01	4.1255e+01
	Rank	**1**	8	7	3	9	5	4	6	2
F30	mean	1.4444e+06	4.9491e+07	6.9644e+05	6.4946e+05	2.5364e+09	1.9028e+06	9.5586e+05	8.6326e+05	**5.8975e+05**
	std	5.9124e+05	3.2112e+07	8.7858e+04	5.6655e+04	3.0194e+08	8.5054e+05	1.4418e+05	9.0146e+04	1.2009e+04
	Rank	6	8	3	2	9	7	5	4	**1**
Count		9	0	1	6	0	0	0	0	**13**
Ave.Rank		3.38	7.72	5.10	2.86	9.00	4.79	5.41	4.86	**1.86**
Total.Rank		3	8	6	2	9	4	7	5	**1**

Table 5
Optimization results and comparison for CEC-2022 test functions (the dimension $m = 10, 20$).

Function	Metric	MPA	IGWO	HBO	RLHBO	MPA	IGWO	HBO	RLHBO
		Dimension=10				Dimension=20			
F1	mean	3.000e+02	3.000e+02	3.000e+02	**3.000e+02**	3.000e+02	3.002e+02	2.081e+03	**3.000e+02**
	std	9.982e−12	1.400e−03	4.100e−03	8.038e−14	7.100e−03	1.756e−01	1.361e+03	2.522e−11
	Rank	2	4	3	**1**	2	3	4	**1**
F2	mean	**4.010e+02**	4.020e+02	4.078e+02	4.073e+02	**4.269e+02**	4.482e+02	4.480e+02	4.481e+02
	std	2.436e+00	2.778e+00	2.098e+00	2.328e+00	2.562e+01	1.688e+00	2.082e+00	1.548e+00
	Rank	**1**	2	4	3	**1**	4	3	2
F3	mean	6.001e+02	6.001e+02	6.000e+02	**6.000e+02**	6.003e+02	6.000e+02	**6.000e+02**	**6.000e+02**
	std	7.000e−04	8.000e−04	8.716e−06	9.282e−14	2.700e−02	1.135e−10	0.000e+00	0.000e+00
	Rank	3	4	2	**1**	4	3	**1**	**1**
F4	mean	8.090e+02	**8.056e+02**	8.274e+02	8.102e+02	8.463e+02	**8.188e+02**	9.080e+02	8.318e+02
	std	3.433e+00	3.259e+00	5.169e+00	4.251e+00	1.722e+01	5.164e+00	4.666e+00	1.021e+01
	Rank	2	**1**	4	3	3	**1**	4	2
F5	mean	9.001e+02	9.000e+02	9.000e+02	**9.000e+02**	9.409e+02	**9.000e+02**	9.016e+02	9.011e+02
	std	4.937e−01	7.838e−06	8.570e−02	1.467e−13	4.072e+01	6.563e−14	6.034e−01	5.958e−01
	Rank	4	2	3	**1**	4	**1**	3	2
F6	mean	**1.800e+03**	4.023e+03	3.273e+03	1.805e+03	**1.827e+03**	7.942e+03	2.626e+05	1.844e+03
	std	1.168e−01	1.556e+03	1.428e+03	2.810e+00	1.311e+01	5.105e+03	2.023e+05	1.947e+01
	Rank	**1**	4	3	2	**1**	3	4	2
F7	mean	2.007e+03	2.015e+03	2.001e+03	**2.000e+03**	2.031e+03	2.026e+03	2.027e+03	**2.026e+03**
	std	8.829e+00	9.958e+00	3.748e+00	8.695e−07	6.444e+00	5.043e+00	6.120e+00	2.490e+00
	Rank	3	4	2	**1**	4	2	3	**1**
F8	mean	2.203e+03	2.216e+03	2.212e+03	**2.202e+03**	2.223e+03	2.226e+03	2.227e+03	**2.222e+03**
	std	5.926e+00	1.009e+01	8.806e+00	1.617e+00	8.756e−01	5.153e+00	1.286e+00	6.388e−01
	Rank	2	4	3	**1**	2	3	4	**1**
F9	mean	2.529e+03	2.529e+03	2.529e+03	**2.529e+03**	2.481e+03	2.481e+03	2.481e+03	**2.481e+03**
	std	2.689e−08	8.302e−14	4.587e−09	0.000e+00	1.548e−10	2.661e−09	7.393e−05	0.000e+00
	Rank	4	2	3	**1**	2	3	4	**1**
F10	mean	2.511e+03	2.524e+03	2.477e+03	**2.471e+03**	2.501e+03	2.500e+03	2.407e+03	**2.405e+03**
	std	3.247e+01	4.535e+01	5.270e+01	3.464e+01	1.833e−01	2.040e−02	1.151e+01	3.809e+00
	Rank	4	3	2	**1**	4	3	2	**1**
F11	mean	**2.600e+03**	2.871e+03	2.894e+03	2.751e+03	**2.875e+03**	2.900e+03	2.900e+03	2.900e+03
	std	1.000e−04	9.016e+01	2.690e+01	3.367e−01	1.892e+02	5.458e−09	3.713e−13	2.625e−13
	Rank	**1**	3	4	2	**1**	4	3	2
F12	mean	**2.861e+03**	2.861e+03	2.862e+03	2.862e+03	2.946e+03	**2.936e+03**	2.940e+03	2.939e+03
	std	1.586e+00	1.528e+00	1.166e+00	7.205e−01	1.092e+01	5.334e+00	2.836e+00	1.975e+00
	Rank	**1**	2	4	3	4	**1**	3	2
Count		4	1	0	**7**	3	3	1	**6**
Ave.Rank		2.18	3.00	3.09	**1.73**	2.67	2.58	3.17	**1.50**
Total.Rank		2	3	4	**1**	3	2	4	**1**

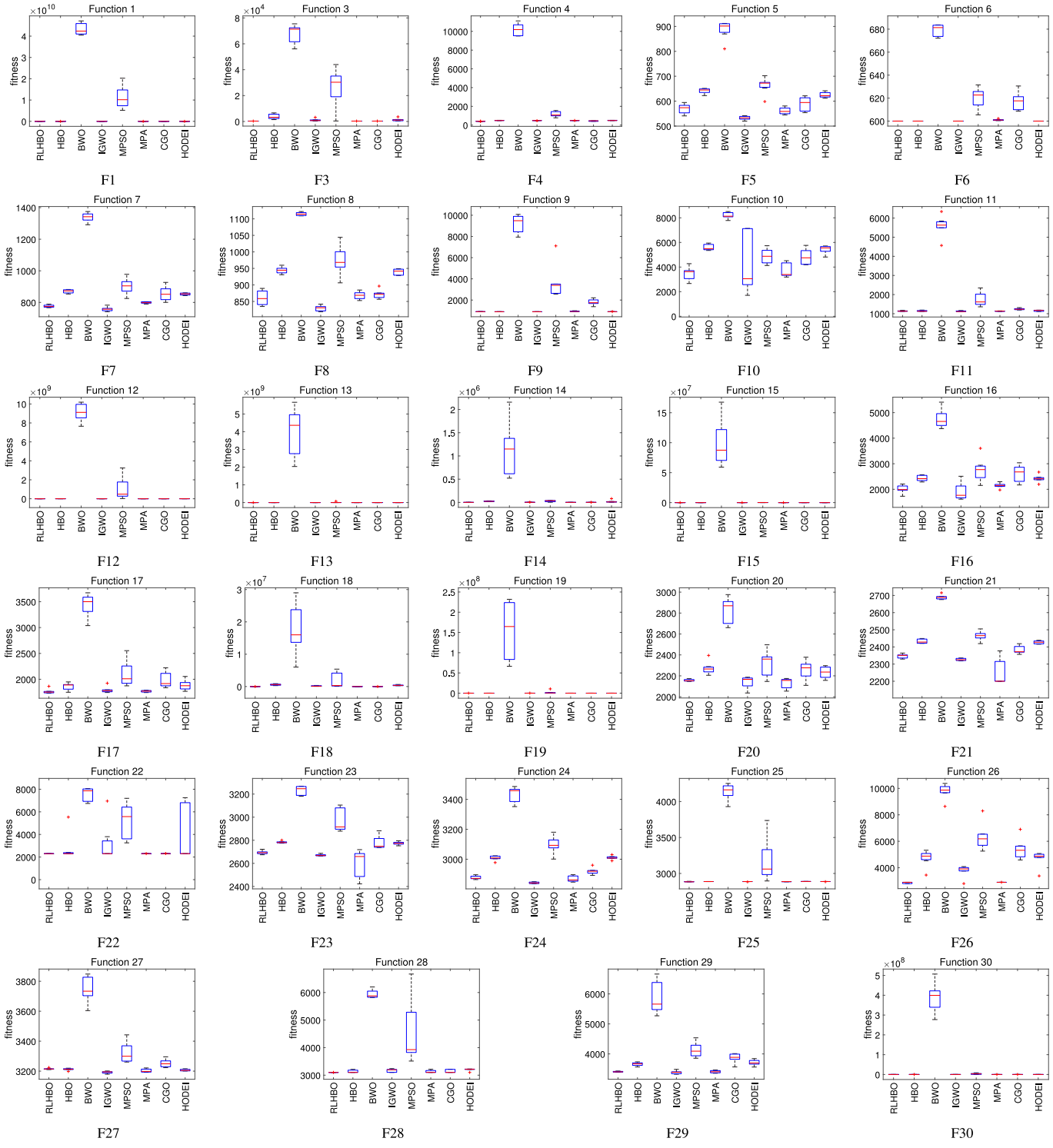


Fig. 6. Boxplot of RLHBO and competitor algorithms in optimization of the CEC-2017 (the dimension m = 30).

algorithms without outliers, while all other algorithms contain outliers. RLHBO demonstrates the best stability and convergence accuracy. The experimental results above demonstrate that RLHBO exhibits better algorithm stability and convergence accuracy.

4.7. Convergence analysis

Convergence is indeed a pivotal metric for assessing the efficacy of optimization algorithms. It provides insight into an algorithm's ability to efficiently navigate the search space, balancing the dual objectives of exploration and exploitation. Adequate exploration ensures the algorithm does not prematurely converge to suboptimal local minima, while effective exploitation guides it towards the global optimum promptly.

To evaluate RLHBO's convergence capability, we conducted experiments utilizing the CEC-2017 test functions (with a dimension of 30). These experiments aimed to analyze RLHBO's convergence across

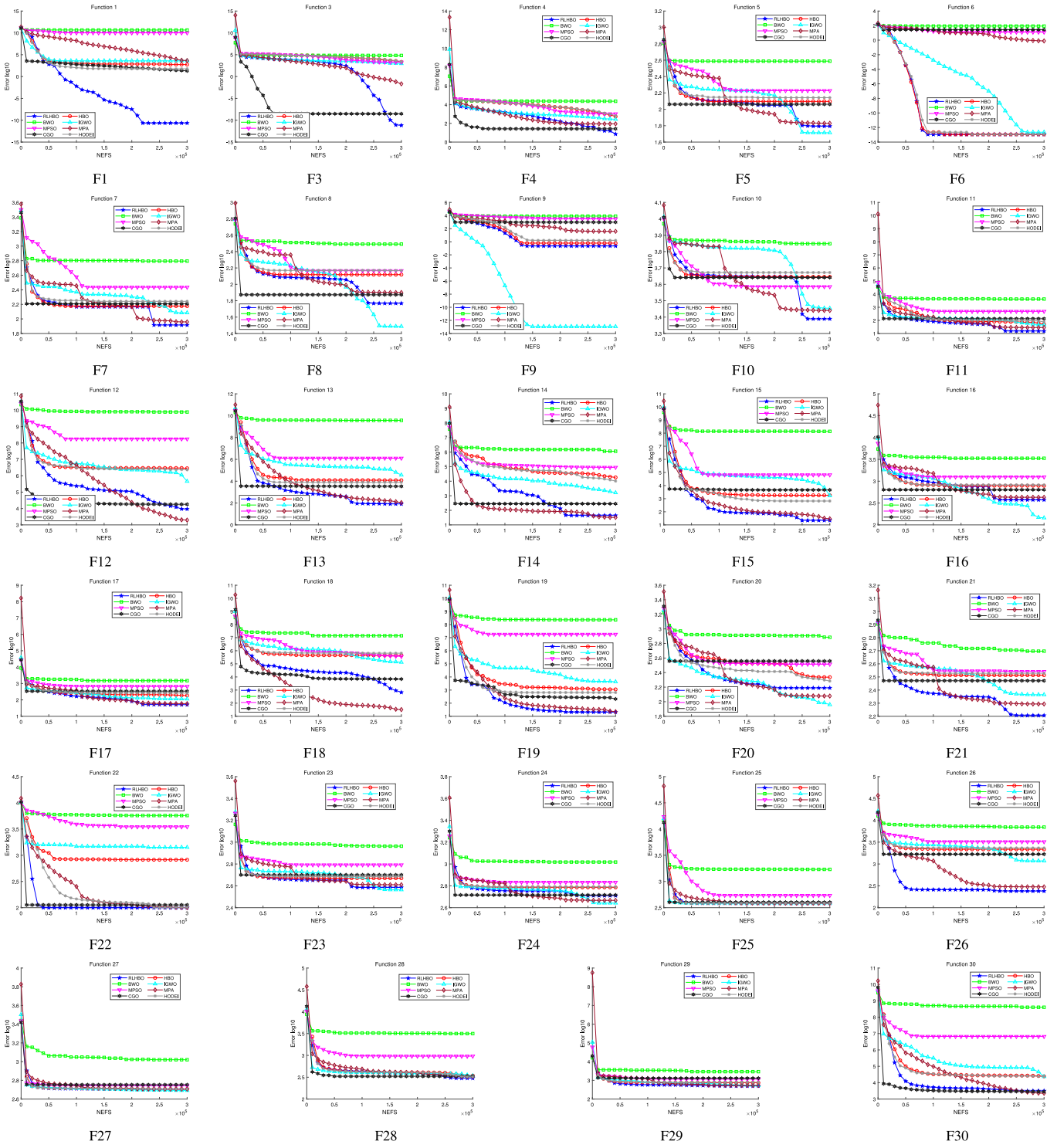


Fig. 7. Convergence curves of RLHBO and its comparison algorithms on the 30-dimensional CEC2017 functions. (The blue polyline represents RLHBO.)

Table 6
Wilcoxon signed rank test results on the 10-dimensional functions from CEC2017.

Compared algorithm	p-value	R^+	R^-	$n/w/t/l$
RLHBO vs. IGWO	7.7321e−05	399.5	35.5	29/24/0/5
RLHBO vs. MPSO	2.1968e−06	435	0	29/29/0/0
RLHBO vs. CGO	3.4681e−05	408.5	26.5	29/25/0/4
RLHBO vs. MPA	2.4826e−03	354.5	80.5	29/22/0/7
RLHBO vs. BWO	2.9631e−06	432.5	2.5	29/28/0/1
RLHBO vs. CEOA	1.0716e−05	424.5	10.5	29/26/2/1
RLHBO vs. HBO	5.0467e−06	433.5	1.5	29/27/2/0
RLHBO vs. HODEI	5.2128e−06	433.5	1.5	29/27/2/0

Table 7
Wilcoxon signed rank test results on the 30-dimensional functions from CEC2017.

Compared algorithm	p-value	R^+	R^-	$n/w/t/l$
RLHBO vs. IGWO	4.8497e−04	378	57	29/21/0/8
RLHBO vs. MPSO	1.7801e−06	435	0	29/29/0/0
RLHBO vs. CGO	3.8367e−05	429.5	5.5	29/28/0/1
RLHBO vs. MPA	2.7050e−02	317.5	117.5	29/22/0/7
RLHBO vs. BWO	1.5120e−06	435	0	29/29/0/0
RLHBO vs. COA	2.3200e−05	412	23	29/24/0/5
RLHBO vs. HBO	3.3078e−06	434.5	0.5	29/28/1/0
RLHBO vs. HODEI	3.4270e−06	434.5	0.5	29/28/1/0

Table 8
Wilcoxon signed rank test results on the 50-dimensional functions from CEC2017.

Compared algorithm	p-value	R^+	R^-	$n/w/t/l$
RLHBO vs. IGWO	5.9589e−03	343.5	91.5	29/18/0/11
RLHBO vs. MPSO	1.9982e−06	435	0	29/29/0/0
RLHBO vs. CGO	3.8127e−06	430	5	29/27/0/2
RLHBO vs. MPA	4.2199e−02	310	125	29/20/0/9
RLHBO vs. BWO	1.8071e−06	435	0	29/29/0/0
RLHBO vs. CEOA	1.4165e−05	417.5	17.5	29/26/0/3
RLHBO vs. HBO	2.3323e−06	435	0	29/29/0/0
RLHBO vs. HODEI	2.1029e−06	435	0	29/29/0/0

various function types, including unimodal, multimodal, hybrid, and composite functions.

Fig. 7 displays the convergence curves of the algorithm proposed in this article and the comparative algorithm for various functions. The y-axis in the figure represents the average error between the actual optimal value searched by the algorithm and the theoretical optimal value of the function, while the x-axis represents the number of function evaluations. Fig. 7 depicts the convergence curves of the proposed RLHBO algorithm and comparison algorithms on the CEC2017 function. Among the functions $f_1, f_6, f_{17}, f_{19}, f_{22}, f_{25}, f_{26}$, and f_{29}, RLHBO achieves convergence in the early stages of the search, demonstrating significantly better convergence speed and accuracy compared to other comparison algorithms. In functions $f_3, f_4, f_7, f_{10}, f_{11}, f_{13}, f_{15}, f_{21}$, and f_{28}, the RLHBO algorithm demonstrates its powerful late-stage search capabilities. During the early and mid stages of these function searches, RLHBO is not the optimal algorithm. From Fig. 7, it can be seen that the sharply declining convergence curve in the later period makes RLHBO the best-performing algorithm. Among the various unimodal, multimodal, hybrid, and composite functions in CEC2017, RLHBO demonstrates better performance and convergence speed compared to HBO. This fully validates the effectiveness of the proposed strategy in enhancing the algorithm's exploration and exploitation capabilities, as well as improving its convergence speed and accuracy.

5. Conclusion

To address the limitations of HBO in search capability and convergence speed, this paper introduces the RLHBO algorithm, which incorporates three innovative strategies to elevate its performance. First, it integrates reinforcement learning algorithms to enhance HBO's search capability through a dynamic control search strategy. Secondly, to accelerate convergence, it employs a dimension self-learning strategy alongside a convergence approach based on search direction similarity. These methods aim to reduce the generation of invalid solutions in the initial search phase and discrete solutions later on, thereby hastening convergence. Additionally, this study adopts a quasi-contrastive learning-based technique for population initialization to improve the initial population's quality and diversity, thus strengthening the algorithm's global optimization capacity. Empirical evaluations using the complex functions of CEC2017 and CEC2022 confirm RLHBO's superior performance over HBO and other modern optimization algorithms in optimization efficacy and convergence velocity.

In this work, we demonstrate the significant potential of using reinforcement learning to address the exploitation and exploration challenges of MAs, thereby enhancing the performance of HBO algorithms. However, despite RLHBO's advancements in search efficiency and convergence acceleration, RLHBO encounters certain constraints. For instance, its effectiveness heavily depends on the tuning of reinforcement learning model parameters, like the learning rate and reward function design, potentially affecting its adaptability and sturdiness. The significance of the initial population's quality and the imperative to balance complexity against computational expenses are also emphasized, signifying domains necessitating further investigation and refinement.

Considering these limitations, our future work will include exploring more effective mechanisms for adaptive parameter adjustment to reduce dependence on parameter settings; developing efficient methods for generating initial populations to enhance algorithm performance in handling high-dimensional and complex optimization problems; and reducing the computational cost to increase feasibility in practical applications. Furthermore, we plan to investigate the adoption of more advanced reinforcement learning algorithms to further improve RLHBO's performance and applicability.

CRediT authorship contribution statement

Xuesen Ma: Writing – review & editing, Supervision, Funding acquisition, Conceptualization. **Zhineng Zhong:** Writing – original draft, Software, Methodology, Investigation, Data curation. **Yangyu Li:** Resources, Funding acquisition. **Dacheng Li:** Resources, Funding acquisition. **Yan Qiao:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the Hefei Municipal Natural Science Foundation, China (No. 2022015).

Table 9
Results of the Wilcoxon signed-rank test for HBO with the proposed strategy versus HBO on CEC2017 in 10 dimensions.

Function	RLHBO vs. HBO	HBO1 vs. HBO	HBO2 vs. HBO	HBO3 vs. HBO	HBO1+2 vs. HBO
F1	+	+	+	+	+
F3	+	+	+	+	+
F4	+	+	+	+	+
F5	+	+	+	+	+
F6	+	−	+	+	+
F7	+	+	+	−	+
F8	+	+	+	+	+
F9	≈	≈	≈	≈	≈
F10	+	+	+	−	+
F11	+	+	+	+	+
F12	+	+	+	+	+
F13	+	−	+	−	+
F14	+	+	+	−	+
F15	+	+	+	+	+
F16	+	−	−	−	+
F17	+	+	+	+	+
F18	+	+	+	+	+
F19	+	+	+	−	+
F20	≈	≈	−	≈	≈
F21	+	+	−	−	+
F22	+	−	−	−	+
F23	+	+	+	−	+
F24	+	+	+	+	+
F25	+	+	−	−	−
F26	+	+	+	+	+
F27	+	+	+	+	+
F28	+	−	+	+	+
F29	+	+	+	−	+
F30	+	+	+	+	+
n/+/≈/−	29/27/2/0	29/22/2/5	29/22/1/6	29/17/2/10	29/26/2/1

References

[1] W. Li, P. Liang, B. Sun, Y. Sun, Y. Huang, Reinforcement learning-based particle swarm optimization with neighborhood differential mutation strategy, Swarm Evol. Comput. 78 (2023) 101274.

[2] M. Abdel-Basset, R. Mohamed, M. Jameel, M. Abouhawwash, Spider wasp optimizer: A novel meta-heuristic optimization algorithm, Artif. Intell. Rev. (2023) 1–64.

[3] L. Abualigah, A. Diabat, A comprehensive survey of the grasshopper optimization algorithm: results, variants, and applications, Neural Comput. Appl. 32 (19) (2020) 15533–15556.

[4] L. Abualigah, Group search optimizer: a nature-inspired meta-heuristic optimization algorithm with its results, variants, and applications, Neural Comput. Appl. 33 (7) (2021) 2949–2972.

[5] F.S. Gharehchopogh, Advances in tree seed algorithm: A comprehensive survey, Arch. Comput. Methods Eng. 29 (5) (2022) 3281–3304.

[6] S.B. Joseph, E.G. Dada, A. Abidemi, D.O. Oyewola, B.M. Khammas, Metaheuristic algorithms for PID controller parameters tuning: Review, approaches and open problems, Heliyon (2022).

[7] T. Dokeroglu, A. Deniz, H.E. Kiziloz, A comprehensive survey on recent metaheuristics for feature selection, Neurocomputing 494 (2022) 269–296.

[8] Y. Shi, Particle swarm optimization, IEEE Connect. 2 (1) (2004) 8–13.

[9] G. Karafotias, A.E. Eiben, M. Hoogendoorn, Generic parameter control with reinforcement learning, in: Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, 2014, pp. 1319–1326.

[10] R. Storn, K. Price, Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces, J. Global Optim. 11 (1997) 341–359.

[11] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, Adv. Eng. Softw. 69 (2014) 46–61.

[12] D. Simon, Biogeography-based optimization, IEEE Trans. Evol. Comput. 12 (6) (2008) 702–713.

[13] Q. Askari, M. Saeed, I. Younas, Heap-based optimizer inspired by corporate rank hierarchy for global optimization, Expert Syst. Appl. 161 (2020) 113702.

[14] A. Alorf, A survey of recently developed metaheuristics and their comparative analysis, Eng. Appl. Artif. Intell. 117 (2023) 105622.

[15] D.S. AbdElminaam, E.H. Houssein, M. Said, D. Oliva, A. Nabil, An efficient heap-based optimizer for parameters identification of modified photovoltaic models, Ain Shams Eng. J. 13 (5) (2022) 101728.

[16] Y.Ç. Kuyu, F. Vatansever, Heap-based optimizer embedded with search strategies applied to high-order analog filter designs: a comparative study with up-to-date metaheuristics, Neural Comput. Appl. 35 (2) (2023) 1447–1467.

[17] A.A. Ewees, M.A. Al-qaness, L. Abualigah, M. Abd Elaziz, HBO-LSTM: Optimized long short term memory with heap-based optimizer for wind power forecasting, Energy Convers. Manage. 268 (2022) 116022.

[18] A. Seyyedabbasi, R. Aliyev, F. Kiani, M.U. Gulle, H. Basyildiz, M.A. Shah, Hybrid algorithms based on combining reinforcement learning and metaheuristic methods to solve global optimization problems, Knowl.-Based Syst. 223 (2021) 107044.

[19] H. Samma, C.P. Lim, J.M. Saleh, A new reinforcement learning-based memetic particle swarm optimizer, Appl. Soft Comput. 43 (2016) 276–297.

[20] Z. Yan, J. Yan, Y. Wu, S. Cai, H. Wang, A novel reinforcement learning based tuna swarm optimization algorithm for autonomous underwater vehicle path planning, Math. Comput. Simulation 209 (2023) 55–86.

[21] M. Ghetas, M. Issa, A novel reinforcement learning-based reptile search algorithm for solving optimization problems, Neural Comput. Appl. (2023) 1–36.

[22] F. Wang, X. Wang, S. Sun, A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization, Inform. Sci. 602 (2022) 298–312.

[23] A. Seyyedabbasi, A reinforcement learning-based metaheuristic algorithm for solving global optimization problems, Adv. Eng. Softw. 178 (2023) 103411.

[24] S. Rahnamayan, H.R. Tizhoosh, M.M. Salama, Quasi-oppositional differential evolution, in: 2007 IEEE Congress on Evolutionary Computation, IEEE, 2007, pp. 2229–2236.

[25] V. Basetti, S.S. Rangarajan, C. Kumar Shiva, S. Verma, R.E. Collins, T. Senjyu, A quasi-oppositional heap-based optimization technique for power flow analysis by considering large scale photovoltaic generator, Energies 14 (17) (2021) 5382.

[26] X. Zhang, S. Wen, Heap-based optimizer based on three new updating strategies, Expert Syst. Appl. 209 (2022) 118222.

[27] R.M. Rizk-Allah, I.M. Eldesoky, E.A. Aboali, S.M. Nasr, Heap-based optimizer algorithm with chaotic search for nonlinear programming problem global solution, Int. J. Comput. Intell. Syst. 16 (1) (2023) 149.

[28] A.M. Shaheen, A.M. Elsayed, A.R. Ginidi, R.A. El-Sehiemy, E. Elattar, A heap-based algorithm with deeper exploitative feature for optimal allocations of distributed generations with feeder reconfiguration in power distribution networks, Knowl.-Based Syst. 241 (2022) 108269.

[29] Y. Song, Y. Wu, Y. Guo, R. Yan, P.N. Suganthan, Y. Zhang, W. Pedrycz, Y. Chen, S. Das, R. Mallipeddi, et al., Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities, 2023, arXiv preprint arXiv:2308.13420.

[30] W. Luo, X. Yu, Reinforcement learning-based modified cuckoo search algorithm for economic dispatch problems, Knowl.-Based Syst. 257 (2022) 109844.

[31] Z. Hu, X. Yu, Reinforcement learning-based comprehensive learning grey wolf optimizer for feature selection, Appl. Soft Comput. 149 (2023) 110959.

[32] Y. Tian, X. Li, H. Ma, X. Zhang, K.C. Tan, Y. Jin, Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization, IEEE Trans. Emerg. Top. Comput. Intell. (2022).

[33] F.L. Lewis, D. Vrabie, K.G. Vamvoudakis, Reinforcement learning and feedback control: Using natural decision methods to design optimal adaptive controllers, IEEE Control Syst. Mag. 32 (6) (2012) 76–105.

[34] D. Lee, S.J. Lee, S.C. Yim, Reinforcement learning-based adaptive PID controller for DPS, Ocean Eng. 216 (2020) 108053.
[35] B. Song, Z. Wang, L. Zou, On global smooth path planning for mobile robots using a novel multimodal delayed PSO algorithm, Cogn. Comput. 9 (1) (2017) 5–17.
[36] G. Qian, S. Sural, Y. Gu, S. Pramanik, Similarity between Euclidean and cosine angle distance for nearest neighbor queries, in: Proceedings of the 2004 ACM Symposium on Applied Computing, 2004, pp. 1232–1237.
[37] S. Rahnamayan, H.R. Tizhoosh, M.M. Salama, Opposition-based differential evolution, IEEE Trans. Evol. Comput. 12 (1) (2008) 64–79.
[38] W. Long, J. Jiao, X. Liang, S. Cai, M. Xu, A random opposition-based learning grey wolf optimizer, IEEE Access 7 (2019) 113810–113825.
[39] Z. Wang, L. Huang, S. Yang, D. Li, D. He, S. Chan, A quasi-oppositional learning of updating quantum state and Q-learning based on the dung beetle algorithm for global optimization, Alex. Eng. J. 81 (2023) 469–488.
[40] A. Faramarzi, M. Heidarinejad, S. Mirjalili, A.H. Gandomi, Marine predators algorithm: A nature-inspired metaheuristic, Expert Syst. Appl. 152 (2020) 113377.
[41] S. Talatahari, M. Azizi, Chaos game optimization: a novel metaheuristic algorithm, Artif. Intell. Rev. 54 (2021) 917–1004.
[42] C. Zhong, G. Li, Z. Meng, Beluga whale optimization: A novel nature-inspired metaheuristic algorithm, Knowl.-Based Syst. 251 (2022) 109215.
[43] Y.-w. Jia, X.-t. Chen, C.-b. Yao, X. Li, CEO election optimization algorithm and its application in constrained optimization problem, Soft Comput. 27 (11) (2023) 7363–7400.
[44] M. Dehghani, Z. Montazeri, E. Trojovská, P. Trojovskỳ, Coati optimization algorithm: A new bio-inspired metaheuristic algorithm for solving optimization problems, Knowl.-Based Syst. 259 (2023) 110011.
[45] M.D. Phung, Q.P. Ha, Motion-encoded particle swarm optimization for moving target search using UAVs, Appl. Soft Comput. 97 (2020) 106705.
[46] M.H. Nadimi-Shahraki, S. Taghian, S. Mirjalili, An improved grey wolf optimizer for solving engineering problems, Expert Syst. Appl. 166 (2021) 113917.
[47] G. Wu, R. Mallipeddi, P.N. Suganthan, Problem Definitions and Evaluation Criteria for the CEC 2017 Competition on Constrained Real-Parameter Optimization, Technical Report, National University of Defense Technology, Changsha, Hunan, PR China and Kyungpook National University, Daegu, South Korea and Nanyang Technological University, Singapore, 2017.
[48] D. Yazdani, J. Branke, M.N. Omidvar, X. Li, C. Li, M. Mavrovouniotis, T.T. Nguyen, S. Yang, X. Yao, IEEE CEC 2022 competition on dynamic optimization problems generated by generalized moving peaks benchmark, 2021, arXiv preprint arXiv:2106.06174.
[49] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, Swarm Evol. Comput. 1 (1) (2011) 3–18.